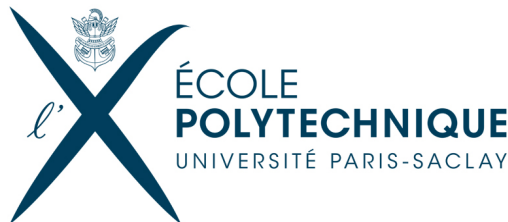


- 3A RESEARCH INTERNSHIP -

Uncertainty estimation and OOD detection with deep learning diffusion models and SURE

Matthieu DESTRADE

5th of May, 2024 - 23rd of August, 2024



DIX, École polytechnique - CVN, CentraleSupélec - ICB, NAIST

Internship tutors: Dr. Hugues Talbot (CS) and Dr. Yoshito Otake (NAIST)
Referent professor: Dr. Steve Oudot (X)

Acknowledgement

I would like to express my sincere thanks to **Professor Talbot** from Centrale Supélec and **Professor Otake** from the NAIST for the guidance and advice they provided me during the beginning of my internship. I am also very grateful to **Yi Gu** for his help on diverse matters, whether they were directly related to my work or not.

Contents

1	Abstract	4
2	Introduction	5
3	Background	7
3.1	Denoising Diffusion Probabilistic models (DDPM)	7
3.2	Stein's Unbiased Risk Estimate (SURE)	8
3.3	Out-of-Distribution (OOD) detection	9
4	Methods	11
4.1	SUREdiff	11
4.2	Implementation	12
4.3	Generation quality	13
4.4	OOD detection	15
4.5	Theoretical guarantees	15
5	Experiments	18
5.1	Generation quality	18
5.2	OOD detection	19
6	BMC estimation	25
6.1	Task	25
6.2	Uncertainty metrics	29
6.2.1	Variance	29
6.2.2	SUREdiff	29
6.2.3	MCDropOut	30
6.2.4	BayesDiff (BD)	30
6.2.5	Image translation	31
6.2.6	Test time augmentation (TTA)	31
6.3	Entropy	32
6.4	Results	32
6.4.1	Pixel level	32
6.4.2	Image level	35
6.4.3	Single generation	35
6.4.4	Mean of several generation	38
6.5	Failure detection	38
6.6	Accuracy improvement using the uncertainties	39
7	Conclusion	40
8	References	41
9	Appendix	44

1 Abstract

In a number of fields, such as medicine or autonomous driving, the estimation of the **uncertainty** of the results of deep learning algorithms is a fundamental task to prevent harmful errors. In this internship, we aim at evaluating medical images generated with **diffusion** [HJA20], and more specifically image translation [Sah+22; Li+23]. Image translation is the process of generating an image based on another one. We wish to be able to detect defects in the generated image as well as in the input image based on the generation uncertainty.

Previous works have studied uncertainty estimation with the development of **Bayesian models** [Mac92; Foo+19; Wen+20; Mal+20]. Such methods usually require heavy computations and come at the cost of a loss of model accuracy. In this report, we propose a more lightweight uncertainty estimation method based on the SURE estimator [Edu+20] that is applicable to any pretrained diffusion network: **SUREdiff**.

In order to test the initial performance of SUREdiff, we perform several **out-of-the-distribution** (OOD) detection tasks inspired by a reconstruction-based approach [BA24], but where uncertainty estimation is used rather than reconstruction error. On these tasks, we obtain very promising performances, reaching **100% accuracy**.

In the remainder of the internship, we shall focus on applying SUREdiff to **image translation** models and evaluating its capabilities. We are especially interested in OOD detection with respect to input images, and in the correlation between the uncertainties and errors of medical measures performed using the generated images. On such tasks, we will compare it to other uncertainty estimation methods, especially [Kou+24], which uses last-layer Laplace approximation to estimate the uncertainty of diffusion models.

2 Introduction

Uncertainty estimation of the results of deep learning networks is a critical task for a number of applications of machine learning. It is essential to ensure the reliability of deep learning models in fields that require results whose safety has to be quantified. Indeed, when detecting obstacles for autonomous vehicles, diagnosing pathologies in medical images, or finding defective products in the industry, on top of qualifying the input data according to its training, the model needs to be able to distinguish new scenarios, data it has never been exposed to, and for which results are very uncertain. Such a capability would, for instance, allow systems to call for human assistance and prevent errors that could otherwise cause accidents or even deaths.

A main avenue to estimate the uncertainty associated with a deep learning model is through the use of **Bayesian models**, which aim at learning the distribution of the results they output rather than just a deterministic value. Several different types of Bayesian models exist. The main methods are drawing samples with MCMC [Mac92] from the results of a Bayesian neural network [GL06], Laplace methods [Foo+19] in which the uncertainty is computed with the curvature of the model, and variational inference [Wen+20] in which the model learns the distribution of its own parameters. However, these approaches are computationally expensive and several approximation methods have been proposed. [GG16] showed that any model using dropout can be converted into a Bayesian one, as it is possible to get such a model to output a distribution by randomly dropping weights over several of its evaluations. Moreover, [Lav+20] explores how to calibrate such methods to improve results and gives an estimate of miscalibration. [Kha+18] presents a way to optimize variational inference networks using only traditional techniques. Finally, [RBB18] uses the last-layer Laplace approximation to compute the uncertainty of any network, and [Kou+24] applies this method to the determination of pixel-wise uncertainty in diffusion models. Some works have also aimed at improving the understanding of what ‘uncertainty’ means: [MG18] differentiates between the different origins of uncertainty in Bayesian models (the data, the model, and the distribution), shows that they can be evaluated using entropy, and develops a network taking this knowledge into account. The paper focuses on classification tasks, but its results can be extended to regression ones with Dirichlet prior networks for regression [Mal+20]. Overall, Bayesian models yield good results but tend to obtain reduced accuracy [Blu+19] compared to more traditional models, have a **heavy computational cost**, and often require a specific model structure.

In the medical field, a task of particular interest is **image generation**, whether it is to directly compute medical measurements with it [Gu23] or to give meaningful insights to doctors [Edu+20]. We therefore wish to compute uncertainties related to tasks of image generation with **denoising diffusion probabilistic models (DDPM)** [HJA20]. More specifically, we are interested in **image translation** with diffusion models [Sah+22; Li+23], with similar settings as in [Gu23], but where the GAN network would be replaced by a diffusion one. Image translation is the task of generating a new image based on the characteristics of an already existing one. In [Gu23], the authors use image translation with a GAN to extract a single bone from an hip x-ray image, and then use the generated image to perform bone mineral density measurements. Working with diffusion models is interesting since image generation with such networks has proven to be very effective in both unconditional [HD05; SE20; Vin11] and conditional image generation [DN21; Kaw+22; Rom+22a].

Our final goal is therefore to qualify the uncertainty of an image generated with image translation, which could allow us to solve two tasks: the **prediction of the error** of medical measures made using the generated image and the **detection of abnormal or out-of-distribution** input images based on the uncertainty of the generation. This leads us to consider methods similar to the Bayesian ones, namely methods that can yield a measure quantifying the uncertainty of the model per generated image, and even per generated pixel. Most of the previously described Bayesian methods cannot be directly used with diffusion models. The particular context of diffusion actually allows us to use a tool for which we have theoretical guarantees, that can be easily used with any pretrained diffusion model, and that has not been applied to them yet: **the Stein Unbiased**

Risk Estimator (SURE). SURE yields an estimate of the Mean Squared Error (MSE) of a denoising estimator without requiring to know what the ground truth, the denoised image, is. This allows us to get an estimate of the error of a model, even on generated images. This estimate of the error can then be used as an uncertainty metric to detect anomalies when generating images. The performances of SURE are confirmed in [Edu+20], in which the authors use it to evaluate the error of an MRI-denoising deep learning model.

At this point in the internship, the work on image translation has not begun yet, and in order to evaluate the initial capabilities of our uncertainty method, it will be tested on **out-of-distribution (OOD) detection tasks** with medical images.

This report therefore describes a new uncertainty estimator for DDPM using the SURE estimator, **SUREdiff**. While the eventual goal of the internship is to apply SUREdiff to image translation, at this point in the internship, we have not been able to use it on such tasks and we will focus on them in the future. To assess its initial performances, we will therefore focus on a simpler task in this report, namely OOD detection using a simple DDPM model and SUREdiff in a reconstruction approach. This report describes a new methodology to do so, along with theoretical guarantees about its results. It also highlights the correlation between uncertainty of generation and **local entropy** of an image, and this knowledge is used to improve the SUREdiff approach. Overall, in practice, this method yields very promising results on this task, achieving 100% OOD detection accuracy on datasets composed of hip x-ray images (reference distribution images) and other types of x-ray images (OOD images).

3 Background

SUREdiff relies on two principal tools: DDPM and SURE. It will be applied to OOD detection tasks.

3.1 Denoising Diffusion Probabilistic models (DDPM)

Diffusion models take inspiration from score-matching models [HD05]. They are first described as Markov chains [HJA20] and later as continuous processes [Son+21]. They allow for the generation of images following the distribution of a training dataset through a forward and a backward diffusion processes.

Let $x \in \mathbb{R}^{c \times h \times \omega}$ represent an image. Typically, c and h are the dimensions of the image, and $\omega = 3$ (color channels). In our case we are working with black and white images, therefore $\omega = 1$.

The forward process consists of diffusing the target data distribution $q(x)$ towards $q_t(x_t)$ for all $t \in [0, T]$, with $q_T(x_T) = \mathcal{N}(0, I)$. The transition distribution is $q_t(x_t|x) = \mathcal{N}(x_t; \alpha_t x, \sigma_t^2 I)$ where $\alpha_t, \sigma_t \in \mathbb{R}_+$.

The stochastic differential equation (SDE) satisfying this transition is:

$$dx_t = f(t)x_t dt + g(t)d\omega_t$$

where ω_t is the Wiener process (Brownian motion), $f(t) = \frac{d \log \alpha_t}{dt}$, and $g(t)^2 = \frac{d\sigma_t^2}{dt} - 2 \frac{d \log \alpha_t}{dt} \sigma_t^2$.

It is possible to reverse this process using another SDE:

$$dx_t = \left(f(t)x_t + \frac{g(t)^2}{\sigma_t} \epsilon_t \right) dt + g(t)d\bar{\omega}_t \quad (1)$$

where $\bar{\omega}_t$ is the reverse-time Wiener process and $\epsilon_t := \epsilon_\theta(x_t, t)$ is the total noise added at time t , estimated by the diffusion neural network parameterized by θ . This network is trained by minimizing:

$$\mathbb{E}_{x \sim q(x), \epsilon \sim \mathcal{N}(0, I), t \sim \mathcal{U}(0, T)} \left(w(t) \|\epsilon_\theta(\alpha_t x + \sigma_t \epsilon, t) - \epsilon\|_2^2 \right)$$

where $w(t)$ is a weighting function. Interestingly, the quantity $-\epsilon_\theta(x_t, t)/\sigma_t$ approximate the data score $\nabla_{x_t} \log q_t(x_t)$.

There also exists an ordinary differential equation describing the reverse diffusion process:

$$\frac{dx_t}{dt} = f(t)x_t + \frac{g(t)^2}{2\sigma_t} \epsilon_t \quad (2)$$

Therefore, in order to sample an image $x_0 \sim q(x)$, it is possible to sample $x_T \sim \mathcal{N}(0, I)$ and to reverse time using one of the equations. It is the backward process.

In practice, the simplest DDPM models use a **U-net** (see Fig.1) to learn ϵ_θ , and a discretized version of (1). They use a sequence $\alpha_1 > \alpha_2 > \dots > \alpha_T$ and train the network using gradient descent steps on

$$\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$$

where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, and x_0 is a training image.

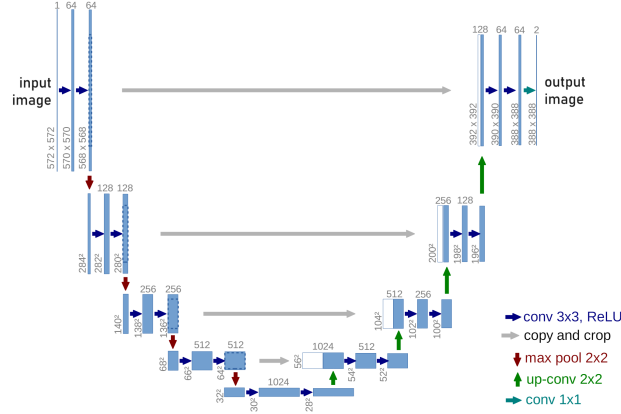


Figure 1: U-net diagram from [RFB15] (with arbitrary parameters)

To generate new images, they sample $x_T \sim \mathcal{N}(0, I)$, and for $t = T, T-1, \dots, 1$ they compute

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

where $z \sim \mathcal{N}(0, I)$ if $t > 1$ and $z = 0$ otherwise, $\sigma_t = \sqrt{\tilde{\beta}_t} = \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \alpha_t} (1 - \alpha_t)}$ (for the implementation of DDPM we use in this project, namely the one proposed by diffusers from Hugging Face [Pla+22]).

DDPM are intrinsically stochastic. An alternative deterministic version, based on the discretization of equation (2), called DDIM, also exists.

3.2 Stein's Unbiased Risk Estimate (SURE)

SURE is an estimator of the mean squared error (MSE) of a denoising estimator that does not require to have access to the real denoised version of the noised data. Let $d \in \mathbb{N}$, $\mu \in \mathbb{R}^d$, $\sigma \in \mathbb{R}_+$ and $g : \mathbb{R}^d \mapsto \mathbb{R}^d$ weakly differentiable. We define: $\forall x \in \mathbb{R}^d, h(x) = x + g(x)$. The SURE estimator of h at $x \in \mathbb{R}^d$ is:

$$SURE_h(x) := d\sigma^2 + \|g(x)\|^2 + 2\sigma^2 \sum_{i=1}^d \frac{\partial}{\partial x_i} g_i(x) = -d\sigma^2 + \|g(x)\|^2 + 2\sigma^2 \sum_{i=1}^d \frac{\partial}{\partial x_i} h_i(x)$$

And we have (the proof is available in the appendix):

$$\mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(SURE_h(x)) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(\|\mu - h(x)\|^2) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(MSE_h(x))$$

For the rest of the report, since we will be working with images in $\mathbb{R}^{d \times d}$, we define, for all (i, j) in $\llbracket 1, d \rrbracket$:

$$h'_{(i,j)} := \frac{\partial}{\partial x_{i,j}} h_{i,j}$$

SURE will therefore be written in the form:

$$SURE_h(x) = -d^2\sigma^2 + \|g(x)\|^2 + 2\sigma^2 \sum_{i,j=1}^d h'_{(i,j)}(x)$$

Since we will be interested in pixel-wise uncertainty estimation, it is interesting to prove that SURE can be adapted to the case where our estimator is $h_{m,n}(x) : x \in \mathbb{R}^d \mapsto x_{m,n} + g(x) \in \mathbb{R}$ where $g : \mathbb{R}^d \mapsto \mathbb{R}$ and $(m, n) \in \mathbb{R}^2$.

We can define:

$$SURE_{h_{m,n}}(x) := -\sigma^2 + g(x)^2 + 2\sigma^2 \frac{\partial h_{m,n}}{\partial x_{i,j}}(x)$$

Let's define: $\forall (i, j) \in \llbracket 1, d \rrbracket^2, \tilde{h}_{i,j} : x \mapsto \begin{cases} h_{m,n}(x) & \text{if } m = i \text{ and } n = j \\ \mu_{i,j} & \text{otherwise} \end{cases}$. We can apply the traditional SURE formula to $\tilde{h}$ which yields:

$$\mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(SURE_{\tilde{h}}(x)) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(\|\mu - \tilde{h}(x)\|^2)$$

and therefore:

$$\begin{aligned} \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}((\mu_{i,j} - h_{i,j}(x))^2) &= \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(-d^2\sigma^2 + \|\tilde{h}(x) - x\|^2 + 2\sigma^2 \sum_{i,j=1}^d \tilde{h}'_{(i,j)}(x)) \\ &= \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(-d^2\sigma^2 + g(x)^2 + (d^2 - 1)\sigma^2 + 2\sigma^2 \frac{\partial h_{m,n}}{\partial x_{i,j}}(x)) \\ &= \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(SURE_{h_{m,n}}(x)) \end{aligned}$$

which proves that the pixel-wise SURE estimator is still an unbiased estimator of the pixel-wise MSE.

3.3 Out-of-Distribution (OOD) detection

OOD detection is the task of detecting samples outside of a reference distribution. Usually, when performing OOD detection, the method used to differentiate between normal and OOD samples does not rely on training with OOD data, partly because this data is often not available. Even if such data is used in an OOD detection method, it is important that this method does not become specific only to the kind of OOD data available and is still able to detect virtually any OOD sample possible.

Two different types of OOD samples, with respect to a reference distribution, can be defined. The ones that originate from a **covariate shift** have a different distribution but a semantic meaning similar to the reference images. The ones that originate from a **semantic shift** have both different distributions and semantic meanings. We will focus on the semantic shift for our OOD detection tasks.

Numerous methods have been developed to perform such tasks using various techniques, which the survey [Yan+24] describes very well. We will summarize the main ones here. We can distinguish between five main approaches:

- **Classification-based approaches**, which originate from [HG18]. In this paper, the authors use the softmax outputs of a classification network to quantify how well the different classes are separated and to what extent the model is familiar with the input data. This approach can be directly applied to networks already trained for classification tasks (post-hoc detection). It can also be applied to specially trained networks with custom datasets that could, for instance, include some OOD samples specifically tailored for OOD detection. For instance, it is the case of [Xie+21], which uses the latent representation of a variational auto-encoder (VAE) to classify in a semi-supervised fashion, but also of [LLS20], whose ODIN method uses the addition of noise to the input data and temperature scaling to improve the detection results. This field of OOD detection is the most active, according to [Yan+24], and yields the best results.
- **Density-based approaches**, which use deep generative models (DGM) to learn the density of the training data and output the likelihood of a given input, as suggested in [Bis94]. Inputs with low likelihood could then be qualified as OOD. However, [Nal+19] showed that DGM tended to assign higher likelihoods to OOD samples than to in-distribution ones, which might be concerning regarding their intrinsic capabilities. If [Wan+20] shows that the fact that higher likelihoods are assigned to OOD samples might not be a sign of intrinsic deficiency, it still mentions that the detection heavily depends on the task, that it seems hard to generalize model behaviors, and that it is necessary to use other metrics than the likelihood when using DGMs. These methods are also computationally expensive.
- **Distance-based approaches**, which are based on the fact that the OOD samples should be far from the training distribution. These approaches first directly used the distances between the input data, such as [Lee+18], which leveraged the distances to the centroids of classes when dealing with a classification task. To try to improve results, other methods also use distances between embeddings of the input data, such as [MW17], which uses the results of intermediate layers of a classifier, or [Blu+19], which applies normalizing flows to transform the results of a layer into samples following a normal distribution. It then uses the results over several layers to compute uncertainty and detect OODs. However, when the embedding comes from a generative model, these methods might be subject to the same drawbacks as the density-based ones, according to the last paper.
- **Reconstruction-based approaches**, which use DGM to reconstruct input images and qualify whether they are OOD using the distance between the reconstruction and the original image. When dealing with images, several image generation models can be used. [Lia+22] uses a GAN relying on an encoder collecting several frequency components of the input image and merging them using attention to generate new images. [Hu+24] shows that latent stable diffusion can improve GAN results on the same task. [BA24] also uses diffusion models to detect OOD satellite images. These methods yield good results, but they are computationally expensive.
- **Bayesian models**, as previously described, aim at learning the distribution of the results they output, their 'uncertainty'. It is then possible to detect OOD data using this uncertainty and a threshold, for instance. These methods usually yield good results, but they are computationally expensive and hard to train correctly.

The method we will implement in this report shares characteristics with reconstruction-based and Bayesian ones. We will evaluate the uncertainty of the reconstruction of a noised input image to qualify it as OOD or not.

4 Methods

4.1 SUREdiff

We adapt the SURE formula to the reverse diffusion process.

For DDPM, when applying the reverse diffusion process, we estimate:

$$\hat{x}_{t-1} = h_t(x_t) + \sigma_t \epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$, $\sigma_t = \sqrt{\tilde{\beta}_t} = \sqrt{\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}(1-\alpha_t)}$ and $h_t(x_t) = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t))$. Moreover, we have, coming from the forward diffusion process:

$$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$. And therefore:

$$\frac{1}{\sqrt{\alpha_t}}x_t = x_{t-1} + \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}}\epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$. This means that:

$$\frac{x_t}{\sqrt{\alpha_t}} \sim \mathcal{N}(x_{t-1}, \frac{1-\alpha_t}{\alpha_t}I)$$

Therefore we can apply SURE to $\tilde{h}_t : x \mapsto h_t(\sqrt{\alpha_t}x) + \sigma_t\epsilon$:

$$\begin{aligned} \mathbb{E}_s \left(\text{SURE}_{\tilde{h}_t} \left(\frac{x_t}{\sqrt{\alpha_t}} \right) \right) &= \mathbb{E}_s \left(-d^2 \frac{1-\alpha_t}{\alpha_t} + \left\| \tilde{h}_t \left(\frac{x_t}{\sqrt{\alpha_t}} \right) - \frac{x_t}{\sqrt{\alpha_t}} \right\|^2 + 2 \frac{1-\alpha_t}{\alpha_t} \sum_{i,j=1}^d (\tilde{h}'_t)_{(i,j)} \left(\frac{x_t}{\sqrt{\alpha_t}} \right) \right) \\ &= \mathbb{E}_s \left(-d^2 \frac{1-\alpha_t}{\alpha_t} + \left\| h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}} + \sigma_t \epsilon \right\|^2 + 2 \frac{1-\alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t) \right) \\ &= \mathbb{E}_s \left(-d^2 \frac{1-\alpha_t}{\alpha_t} + d^2 \tilde{\beta}_t + \left\| h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}} \right\|^2 + 2 \frac{1-\alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t) \right) \\ &= \mathbb{E}_s \left(-d^2 \frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)} + \left\| h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}} \right\|^2 + 2 \frac{1-\alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t) \right) \end{aligned}$$

where $\mathbb{E}_s = \mathbb{E}_{\frac{x_t}{\sqrt{\alpha_t}} \sim \mathcal{N}(x_{t-1}, \frac{1-\alpha_t}{\alpha_t}I), \epsilon \sim \mathcal{N}(0, I)}$

Therefore, iteration wise, if h is the process from which we get a denoised image, we can define:

$$\boxed{\text{SURE}_{diff}(t) = -d^2 \frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)} + \left\| h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}} \right\|^2 + 2 \frac{1-\alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t)}$$

The same reasoning over several iterations of the diffusion process, to go from time step t to time step s ($t > s$), also yields:

$$\begin{aligned} SURE_{diff}(t, s) = & -d^2 \left(\frac{\bar{\alpha}_s - \bar{\alpha}_t}{\bar{\alpha}_t} - (1 - \alpha_{s+1}) \frac{1 - \bar{\alpha}_s}{1 - \bar{\alpha}_{s+1}} \right) + \|h_{s+1}(\tilde{g}_{s+2,t}(x_t)) - \frac{\sqrt{\bar{\alpha}_s}}{\sqrt{\bar{\alpha}_t}} x_t\|^2 \\ & + 2 \frac{\sqrt{\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_s}} \frac{\bar{\alpha}_s - \bar{\alpha}_t}{\bar{\alpha}_t} \sum_{i,j=1}^d \prod_{m=s+1}^t (h'_m)_{(i,j)}(\tilde{g}_{m+1,t}(x_t)) \end{aligned}$$

where $\tilde{g}_{i,j} : x \mapsto (g_i \circ g_{i+1} \circ \dots \circ g_j)(x)$ and $g_i : x \mapsto h_i(x) + \sigma_i \epsilon_i$ where ϵ_i is fixed and has been sampled following $\mathcal{N}(0, I)$.

These two quantities are estimators of the MSE of the diffusion model over a chosen set of denoising iterations for the whole image. It is also possible to define a pixel-wise SUREdiff estimator for one denoising step:

$$\forall (i, j) \in \llbracket 1, d \rrbracket^2, SURE_{pxw-diff}(t)_{(i,j)} = -\frac{(1 - \alpha_t)^2}{\alpha_t(1 - \bar{\alpha}_t)} + \left(h_t(x_t)_{(i,j)} - \frac{(x_t)_{(i,j)}}{\sqrt{\alpha_t}} \right)^2 + 2 \frac{1 - \alpha_t}{\sqrt{\alpha_t}} (h'_t)_{(i,j)}(x_t)$$

The same goes for several denoising steps.

4.2 Implementation

To implement SUREdiff, we used the **diffusers** library of Hugging Face [Pla+22] to train a DDPM model (adapting a code from former CentraleSupélec students). We computed SUREdiff for one denoising step using a simplified version of the above formula, following additional hypothesis. Namely, we supposed that the estimator h_t was sufficiently good, that is to say that $h_t(x_t) \approx x_{t-1} + \sigma_t \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$. Since

$$x_{t-1} - \frac{x_t}{\sqrt{\alpha_t}} + \sigma_t \epsilon = \frac{(1 - \alpha_t)}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \tilde{\epsilon}$$

where $\epsilon, \tilde{\epsilon} \sim \mathcal{N}(0, I)$, we have:

$$\mathbb{E}_s \left(-d^2 \frac{(1 - \alpha_t)^2}{\alpha_t(1 - \bar{\alpha}_t)} + \|h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}}\|^2 \right) \approx 0$$

We verified that it is the case in practice, except for the last denoising step:

$$\mathbb{E}_s \left(-d^2 \frac{(1 - \alpha_t)^2}{\alpha_t(1 - \bar{\alpha}_t)} + \|h_t(x_t) - \frac{x_t}{\sqrt{\alpha_t}}\|^2 \right) < \mathbb{E}_s \left(2 \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t) \right)$$

This gives us a new, more stable estimator:

$$SURE_{diff2}(t) = 2 \frac{1 - \alpha_t}{\sqrt{\alpha_t}} \sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t)$$

as well as its pixel-wise version:

$$\forall (i, j) \in \llbracket 1, d \rrbracket^2, SURE_{pxw-diff2}(t)_{(i,j)} = 2 \frac{1 - \alpha_t}{\sqrt{\alpha_t}} (h'_t)_{(i,j)}(x_t)$$

We can adapt the formulas over several denoising steps in a similar way.

Concerning the computation of the derivatives, two different approaches are possible. If we are interested in the uncertainty at the image level, it is possible ([Edu+20]) to approximate the Jacobian trace of h_t using:

$$\sum_{i,j=1}^d (h'_t)_{(i,j)}(x_t) \approx b^T (h_t(x_t + \epsilon b) - h_t(x_t)) \epsilon^{-1}$$

where b is a d dimensional noise vector drawn from $\mathcal{N}(0, 1)$ and ϵ is a small positive value.

However, this is not possible if we are interested in the uncertainty per pixel, as it will eventually be our case. In that case, it is necessary to compute the partial derivative of the estimator over every pixel of the image, which is not possible in practice because the computation time of a single evaluation of the model is not negligible, and the number of pixels is too high. Therefore, we supposed that pixels that are not too close to each other are **independent** and that we could compute partial derivatives over distant pixels without using several evaluations of the model. More precisely, let's choose $k \in \mathbb{N}$ the minimal distance between independent pixels and define:

$$\forall(m, n) \in \llbracket 0, k-1 \rrbracket^2, \forall(i, j) \in \llbracket 1, d \rrbracket^2, (\tilde{\epsilon}_{m,n})_{i,j} := \begin{cases} \epsilon & \text{if } i \equiv m[k] \text{ and } j \equiv n[k] \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

where ϵ is a small positive value.

We can then compute the derivative with:

$$\forall(i, j) \in \llbracket 1, d \rrbracket^2, (h'_t)_{i,j}(x_t) = (h_t(x_t + \tilde{\epsilon}_{i\%k, j\%k}) - h_t(x_t))_{i,j} \epsilon^{-1}$$

where $\%$ is the modulus operation.

In total, this method thus requires $1 + k^2$ evaluations of the U-net model, since there only exist k^2 different $(\tilde{\epsilon}_{i\%k, j\%k})$.

SUREdiff was implemented in a **parallelizable** approach.

4.3 Generation quality

The first application of SUREdiff is to qualify the **quality** of images generated with a diffusion model. To get uncertainty results for a generation, we compute the derivatives of the model at every denoising step. We can then compute SUREdiff for every iteration (or a subset of iterations) and combine the results by computing the mean uncertainty of the process, for instance. We can also apply our formula to the whole reverse diffusion process (which is the same as multiplying the derivatives of every iteration if we neglect the first part of the formula). We can finally output the image-wise uncertainty and the pixel-wise one.

No matter what approach we choose, when computing uncertainty images with pixel-wise SUREdiff, we notice that the uncertainty is correlated with the intrinsic complexity of the images generated, that is to say, with their **local entropy**. For instance, an image with a lot of sharp edges is harder to generate than a flat one, and its generation is therefore intrinsically more uncertain. This is not very surprising since, given the surroundings of a pixel, the model has to choose between more plausible values for this pixel if it is in a region with a lot

of details than if it is in a flat one. However, this is not linked to the generation quality, since complex images could better correspond to the training images. Actually, this issue seems to also arise in [Kou+24], whereas the authors use a very different way of computing pixel-wise uncertainties for a diffusion model. Indeed, the uncertainty images they show also seem to be correlated with the complexity of the images generated: images labeled "most certain" are always quite flat, and on the contrary, images labeled "most uncertain" have a lot of details. This issue might therefore not be due specifically to our method but be a consequence of what the "uncertainty" of a generative network really is.

A solution might be to **normalize** the uncertainty with a measure of the complexity of the image: its local entropy. The local entropy of a given pixel in a black and white image is defined as:

$$\sum_{i=1}^n p_i \log p_i$$

where p_i is the probability of the pixel having grey-level i and n the total number of grey levels possible (usually 256). This probability can be approximated by computing the frequency of the grey level in a region of interest around the pixel.

This computation can be done using the `entropy` function from `skimage` [Van+14].

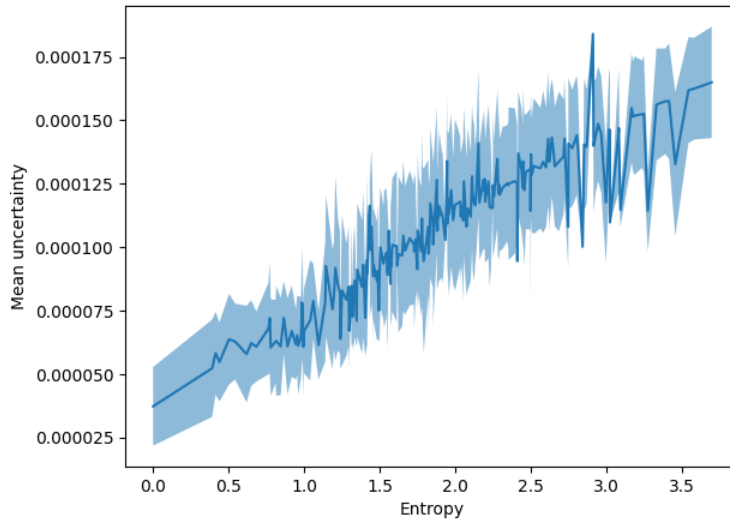


Figure 2: Mean uncertainty per entropy level along with standard deviation per entropy level, computed for the last denoising step of a diffusion model with a dataset of x-ray images

Since we observe that there exists an approximately affine relation between the local entropy of a generated image and the mean uncertainty of the pixels corresponding to a given entropy, we choose to adjust the uncertainties manually so that the mean uncertainty would be **the same for every entropy**. This allows us to give as much weight to the uncertainty in flat regions of the image as to the uncertainty in detailed regions. This adjustment uses the equation of the linear regression of the mean uncertainty with respect to the entropy

to normalize the uncertainties at every entropy level. It depends on the data the model is trained on and on the approach we use to compute the uncertainty. It can be determined by generating several images alongside their uncertainties with the diffusion model.

We also observed a sort of **edge effect** on the borders of the uncertainty images, with very unstable uncertainties. We therefore chose not to use the edge pixels when performing OOD detection.

4.4 OOD detection

SUREdiff also allows for OOD detection, following a method similar to **reconstruction-based** approaches with diffusion [BA24]. The idea behind the method is that the model should know what to do with images that look like the ones it has been trained on and be uncertain with images for which that is not the case. We therefore apply the following procedure:

- We **train** a DDPM model on a set of images following a reference distribution.
- For every image x_0 of a validation set, we choose one or several time steps t corresponding to a state of the diffusion process, and we **add noise** to the image using the formula: $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$. We then compute $SURE_{diff}(t)$ for every time step t .
- Using pixel-wise SUREdiff results, we determine the coefficients necessary for the normalization of the uncertainties with respect to the local entropy of the generated images. We **normalize** the pixel-wise uncertainties accordingly.
- We **sort** the images according to their final uncertainty scores (the mean of the pixel-wise uncertainties over several time steps).
- We **evaluate** the efficiency of the method by computing its AUROC score, calculated by varying the uncertainty threshold separating OOD images from in-distribution ones. We select the best threshold.
- We apply the method used on the validation set to a test set, using the selected threshold.

The **AUROC score** is the area under the Receiving Operating Characteristic curve, namely the curve representing the true positive rate against the false positive rate of a binary classifier depending on a threshold. An AUROC of 0.5 means that the classifier is not better than a random one, and an AUROC of 1.0 means that it is perfect.

4.5 Theoretical guarantees

In this part, we keep the same notations as in 4.2. in the case where $x \in \mathbb{R}^d$.

The equality between the SURE estimator and the MSE is in terms of **expectancy**. However, when generating an image, we have access to only one noisy sample. We could therefore wonder to what extent the unique SURE evaluation is close to the expectancy we are trying to determine.

Let $\tilde{x}$ be sampled from $\mathcal{N}(\mu, \sigma^2)$. Let's suppose that $SURE_h$ is differentiable, and let's note $Jac_{SURE_h}(\cdot)$ its Jacobian matrix. We have, according to the Jensen and mean value inequalities:

$$|\mathbb{E}_x(|\mu - h(x)|^2) - SURE_h(\tilde{x})| \leq \mathbb{E}_x(|SURE_h(x) - SURE_h(\tilde{x})|) \leq \mathbb{E}_x \left(\|x - \tilde{x}\| \sup_{t \in [x, \tilde{x}]} \|Jac_{SURE_h}(t)\| \right)$$

Let's suppose that $\|Jac_{SURE_h}(t)\|$ is bounded by $M \in \mathbb{R}$ on $\mathbb{R}^d$.

$$\begin{aligned}
|\mathbb{E}_x(MSE_h(x)) - SURE_h(\tilde{x})| &\leq M \mathbb{E}_x(\|x - \tilde{x}\|) \\
&\leq M \sqrt{\sum_{i=1}^d \mathbb{E}_x((x_i - \tilde{x}_i)^2)} \quad (\text{Jensen}) \\
&\leq M \sqrt{\sum_{i=1}^d \sigma^2 + \mu_i^2 - 2\mu_i \tilde{x}_i + \tilde{x}_i^2} \\
&\leq M \sqrt{d\sigma^2 + \|\mu - \tilde{x}\|^2}
\end{aligned}$$

Let $\epsilon \in \mathbb{R}$. We have:

$$\begin{aligned}
\mathbb{P}_{\tilde{x}}(|\mathbb{E}_x(MSE_h(x)) - SURE_h(\tilde{x})| < \epsilon) &\geq \mathbb{P}_{\tilde{x}}\left(M\sqrt{d\sigma^2 + \|\mu - \tilde{x}\|^2} < \epsilon\right) \\
&\geq \mathbb{P}_{\tilde{x}}\left(\frac{1}{\sigma^2}\|\mu - \tilde{x}\|^2 < \left(\frac{\epsilon}{\sigma M}\right)^2 - d\right)
\end{aligned}$$

We have $\frac{1}{\sigma^2}\|\mu - \tilde{x}\|^2 \sim \chi_d^2$. Therefore:

$$\boxed{\mathbb{P}_{\tilde{x}}(|\mathbb{E}_x(MSE_h(x)) - SURE_h(\tilde{x})| < \epsilon) \geq \frac{\gamma(d/2, \epsilon^2/(2\sigma^2 M^2) - d/2)}{\Gamma(d/2)}}$$

Therefore, in order to control the precision of our method, we typically have to control the Hessian of our network (since SURE includes its first-order derivatives). Even though our method works with ReLU activation functions (which are used in the U-net), as we will see in Part 6., this concentration inequality implies that it might be possible to get even better results with smoother activation functions. This is something we should test in the future.

A concern we might have concerning our OOD detection method is how SURE compares to directly using the MSE. Indeed, in this particular case **we have access to the ground truth**, the denoised image, and we do not need SURE to have an estimate of the expectancy of the MSE.

We can prove that (the demonstration is detailed in the appendix):

$$\begin{aligned}
\mathbb{V}(MSE_h(x)) - \mathbb{V}(SURE_h(x)) &= \mathbb{E}(-2d\sigma^4 + 4\sigma^2 SURE_h(x) + 8\sigma^2 g(x)^T Jac_g(x)g(x) + 2\sigma^4 \sum_{j=1}^d \sum_{i=1}^d (\frac{\partial g_i}{\partial x_j} + \frac{\partial g_j}{\partial x_i})^2 \\
&\quad + 4\sigma^4 \sum_{j=1}^d g_j(x) \sum_{i=1}^d (\frac{\partial^2 g_i}{\partial x_j \partial x_i}(x) + \frac{\partial^2 g_i}{\partial x_i \partial x_j}(x) + \frac{\partial^2 g_j}{\partial x_i^2}(x)))
\end{aligned}$$

For $d = 1$, this yields:

$$\begin{aligned}
\mathbb{V}(MSE_h(x)) - \mathbb{V}(SURE_h(x)) &= \mathbb{E}(-2\sigma^4 + 4\sigma^2 SURE_h(x) + 8\sigma^2 g(x)^2 g'(x) + 8\sigma^4 g'(x)^2 + 12\sigma^4 g(x)g''(x)) \\
&= \mathbb{E}(2\sigma^4 + 4\sigma^2 g(x)^2 + 8\sigma^4 g'(x) + 8\sigma^2 g(x)^2 g'(x) + 8\sigma^4 g'(x)^2 + 12\sigma^4 g(x)g''(x)) \\
&= \mathbb{E}(\sigma^2 g(x)^2(4 + 8g'(x)) + \sigma^4(2 + 8g'(x) + 8g'(x)^2 + 12g(x)g''(x)))
\end{aligned}$$

With some hypotheses, it can be shown that this quantity is positive. It is reasonable to suppose that for images to which not too much noise has been added, our diffusion model and its derivatives are close to the identity function and its derivatives. We therefore have: for every x , $h(x) \approx x$, $h'(x) \approx 1$, $h''(x) \approx 0$ and $g(x) \approx 0$, $g'(x) \approx 0$, $g''(x) \approx 0$. g and its derivatives are negligible compared to 1. This is verified in practice.

We therefore have:

$$\mathbb{V}(MSE_h(x)) - \mathbb{V}(SURE_h(x)) \approx 2\sigma^4 + 4\sigma^2 g(x)^2 \geq 0$$

With the same hypothesis in dimension d , we also have $Jac_g \approx 0$ and:

$$\mathbb{V}(MSE_h(x)) - \mathbb{V}(SURE_h(x)) \approx 2d\sigma^4 + 4\sigma^2 \|g(x)\|^2 \geq 0$$

Therefore, we can assume that:

$$\boxed{\mathbb{V}(MSE_h(x)) \geq \mathbb{V}(SURE_h(x))}$$

It is therefore **more effective** to evaluate SURE than to evaluate the MSE to access the expectancy of the MSE. Whether or not it will be more effective in practice will, however, also depend on the number of evaluations of the model we perform when computing the partial derivatives to access the value of SURE.

5 Experiments

During the experiments, all the images considered are black and white with 256 grey levels, and their dimensions are 256×256 .

5.1 Generation quality

The first experiments we realized were computing the uncertainties when generating images, and **qualitatively** evaluating whether or not the uncertain images or the uncertain parts of images made sense. We trained a diffusion model for 200 epochs on a dataset of 1791 dental x-ray images, the bitewing dataset. We got the following results:

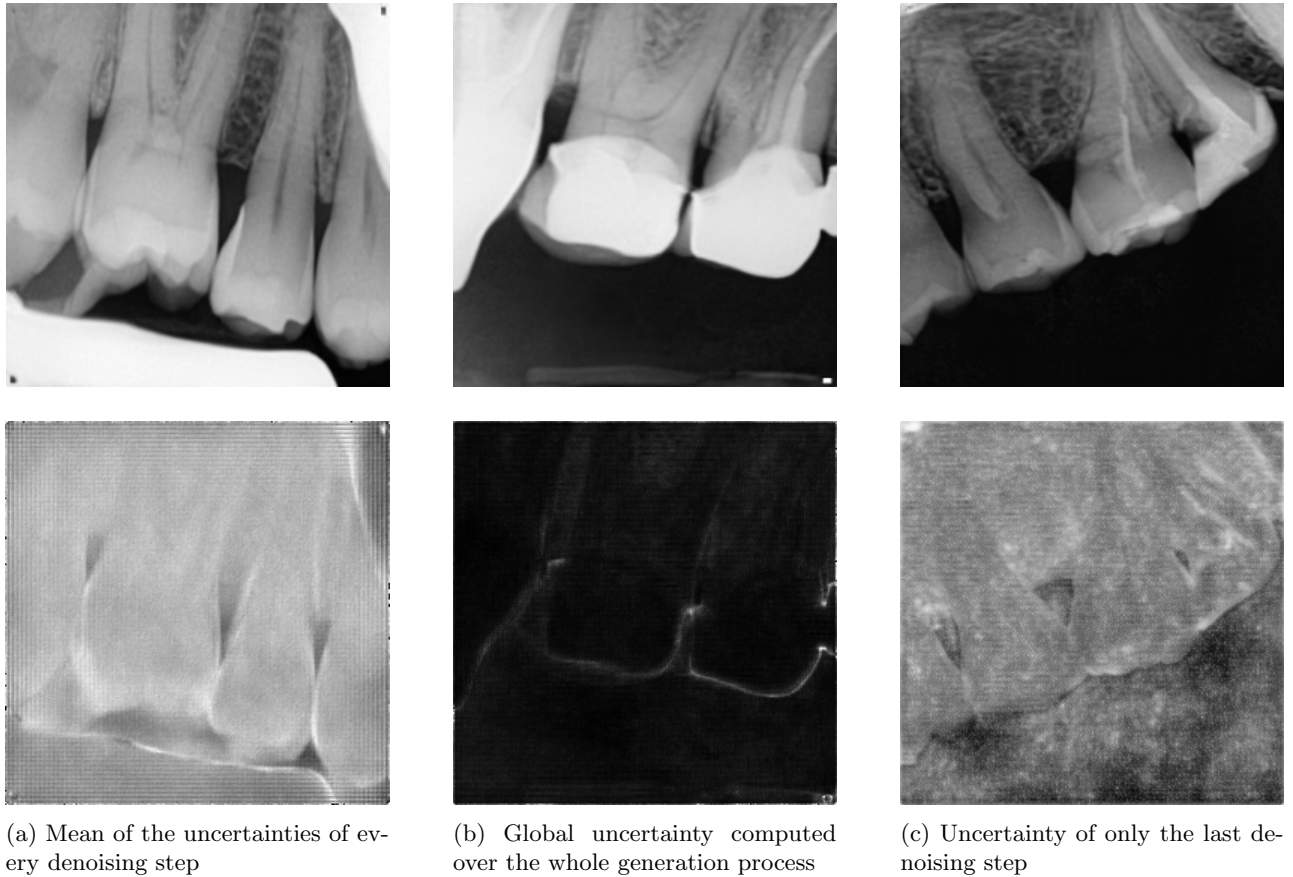


Figure 3: Generated images and their corresponding relative uncertainties, evaluated with different methods (white pixels are the most uncertain, black pixels the least)

No normalization with respect to the entropy was performed for these initial results, and they highlight the **correlation** between regions of high uncertainty and regions of high complexity, high entropy.

However, these initial experiments have **limited scope**. Our final goal is to evaluate the relevance of the computed uncertainties for a quantitative task. Namely, the objective is to perform image translation with a diffusion model and to perform a medical measurement (whose real value is known) on the generated image. We could then evaluate the **correlation** between high uncertainties on the generated image and high measure error. We could also try to assess whether using pixel-wise uncertainties as **weights** for the measure computation could improve the results.

5.2 OOD detection

Performing OOD detection with uncertainties obtained with SUREdiff provides interesting insights about their relevance. Four settings were tested with regards to OOD detection.

Experiment 1

The first experiment involved two datasets. The first one is a data set of 2652 **x-rays of hips** coming from Osaka University. The second one contains 64 **x-rays of hips** and 234 **x-rays of other body parts**. It comes from another source, and the x-ray images were produced with different equipment than the ones of the first dataset, which means that even the distributions of the hip x-ray images are quite **different** in the two datasets. Images extracted from both datasets are available in the appendix.

Applying the method described in 5.4 with time steps 200 and 100, using a diffusion model trained for 400 epochs, and after removing from the calculation the uncertainties coming from the "outlier" pixels (namely the ones with the biggest and lowest uncertainties), we got the following results:

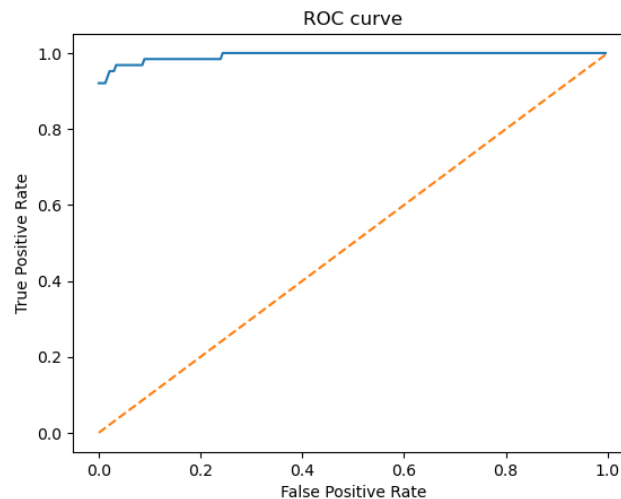
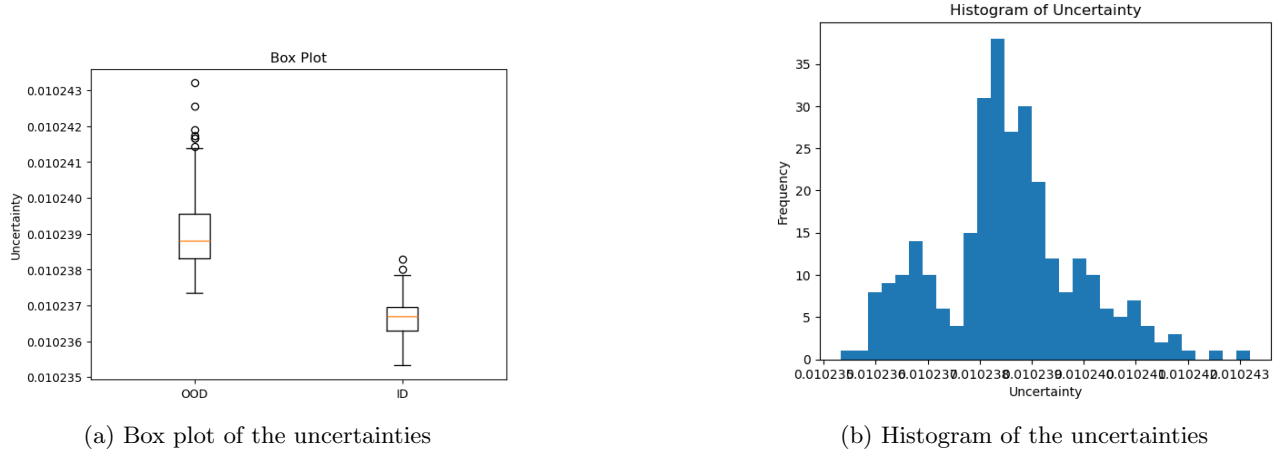


Figure 4: ROC curve of the classifier, AUROC = 0.9936

When setting the best threshold possible, our classifier reaches an **F1 score of 0.9587**, with zero false positive samples and five false negative ones (a sample is positive if it is labeled as ID).



It is interesting to note that the histogram seems to be the union of two approximately **Gaussian distributions** of uncertainties: one with a lower mean for the ID data and another with a higher mean for the OOD data.

It is also interesting to see how our results compare to a UMAP projection of the data. The **Uniform Manifold Approximation and Projection (UMAP)** is a dimensional reduction tool useful to represent high-dimensional data on 2D maps.

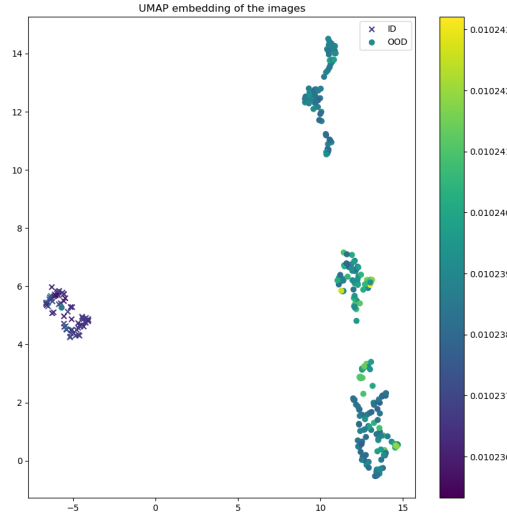


Figure 6: UMAP embedding of the images, colored according to their uncertainty

We observe that the UMAP separates very well the ID data from the OOD one, but it still makes a mistake: one of the OOD images is placed in the center of the cluster of ID images in the UMAP. However, it is **correctly** labeled as more uncertain than most ID images by our method.

Therefore, this experiment shows that our method is able to detect OOD samples and transfer its knowledge

of what an ID sample is even with a different ID dataset than the training one. However, it is not perfect.

Experiment 2

The second experiment involved the dataset containing 64 x-rays of the hips and 234 x-rays of other body parts. The goal was to see if we could reach 100% accuracy when training on data that was closer to the ID data of the test set. Since this dataset is quite small, we performed **cross-validation** to evaluate our method. We therefore separated the set into 4 folds, each of which contained approximately 45 ID train images, 15 ID test images, and 60 OOD test images. The diffusion model was trained for 750 epochs on every training fold.

We computed the uncertainties for timesteps 100, 110, 120, 130, 140, 150, 160, 170, 180, 190, and 200. We got an AUROC of 1.0 for every fold.

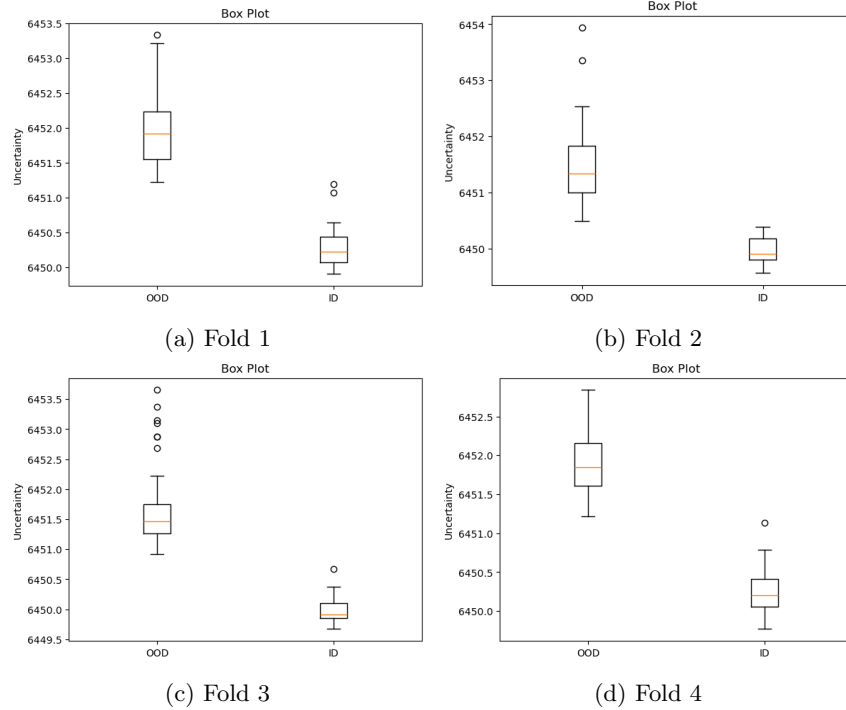


Figure 7: Box plots of the uncertainties

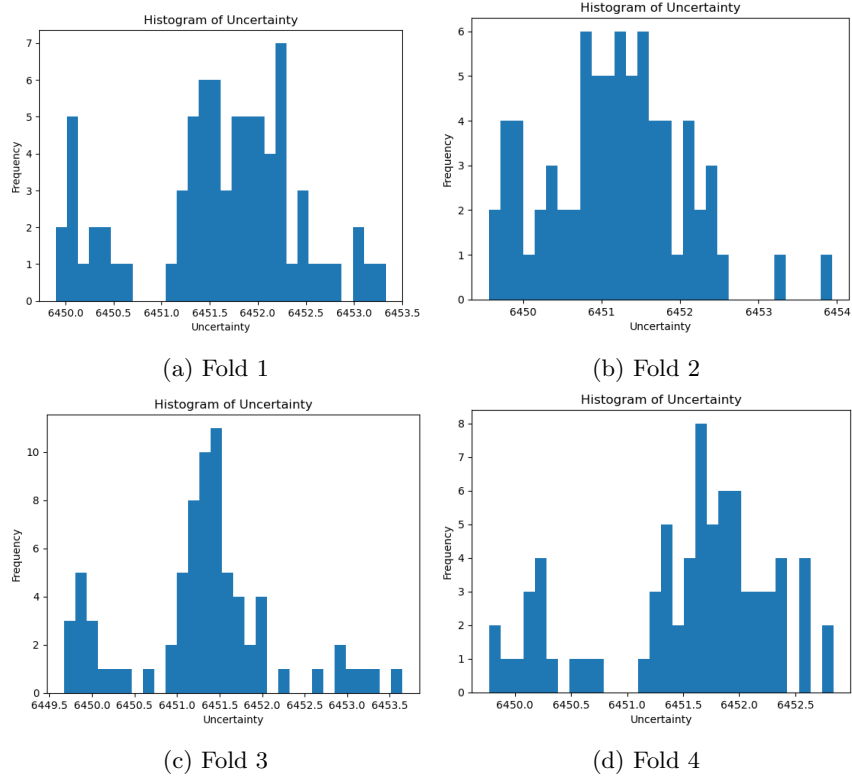


Figure 8: Histograms of the uncertainties

We observe that the uncertainties of the ID and OOD samples are a lot more **separated** than in the first experiment. However, the tails of the two distributions are very close to each other for some folds, and given that the OOD detection process is not deterministic, it could be possible to sometimes get the wrong labels. This issue might be worsened by the fact that we are training a diffusion model with **very few samples** (approximately 45) and that its generalization capabilities are therefore limited. This experiment still proves that, with training data that is more similar to the ID data we want to detect, it is possible to reach 100% accuracy with the SUREdiff OOD detection method.

Experiment 3

To compare the method we propose to one that would be only reconstruction-based and more similar to [BA24], we designed a last OOD detection task with our data. The training set is the dataset from Osaka University, from which we removed 103 images, which will constitute our test ID set. The OOD test set still is the 234 x-rays of other body parts. With these settings, we classified the images using: (1) **SUREdiff scores** computed over time step 0, (2) SUREdiff scores computed over time step 0 **without normalization** with respect to entropy, and (3) the **mean square error** between the reconstruction after time step 0 and the original image. The error computations were averaged over several noised versions of the original image (10 in total) so that

the number of evaluations of the U-net would be **the same for every method** (the derivatives for SUREdiff were computed every 3 pixels). The results are the following:

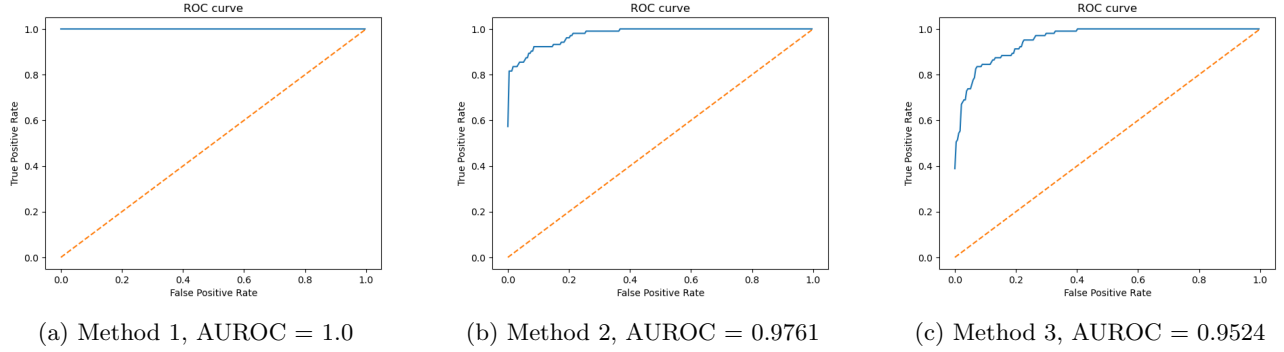


Figure 9: ROC curves of the classifiers

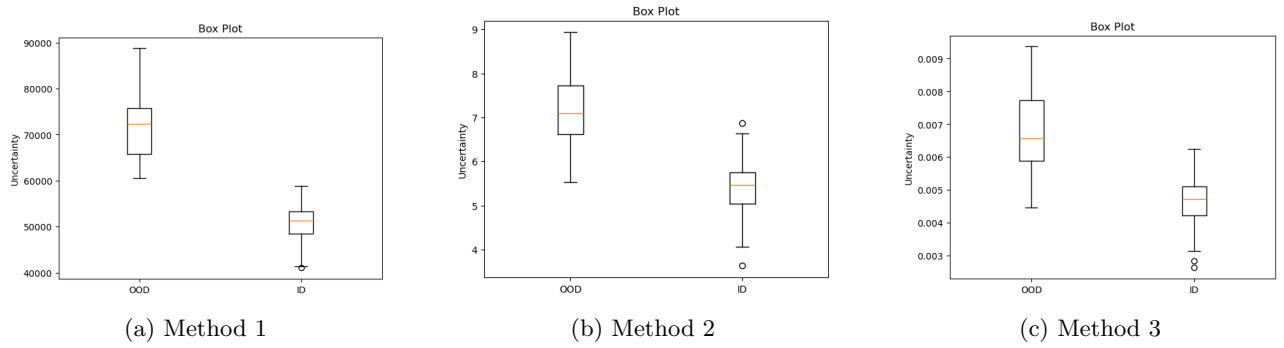


Figure 10: Box plots of the classifiers

These results show the **importance** of normalization with respect to entropy to improve performance. of our method. They are also coherent with the fact that the SURE **variance** is smaller than the MSE one, and show that we can expect better performances using SURE rather than directly using the MSE for OOD detection. This is visible on the error images computed, for instance:

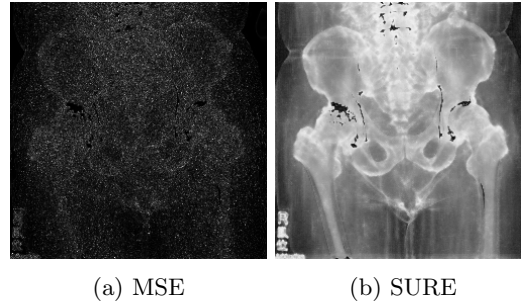


Figure 11: Error via MSE vs error via SURE

Experiment 4

So far, we have evaluated the capabilities of the network to differentiate between images with completely different semantics. It might also be interesting to know how well it performs on **anomaly detection** tasks, that is, tasks where OOD images are in-distribution ones with small defects. So far, we have only been able to realize a small test with artificial OOD images: we modified three images from the bitewing dataset described earlier and tried to separate them from 100 other images extracted from the dataset. Our method correctly assigned them the biggest uncertainties.

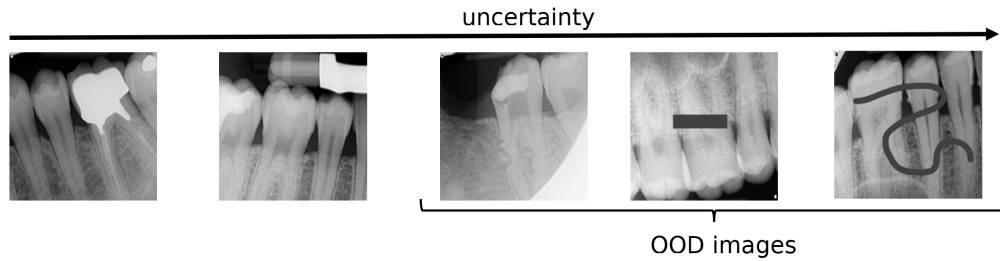


Figure 12: Results of the last experiment

This result is encouraging, but we should still perform experiments on **larger scales** to assess the efficiency of our method for anomaly detection, and with real OOD data (such as broken bones or cancerous tissues amongst healthy ones).

With respect to OOD detection, our final goal is still about the image translation task. We would like to be able to detect if **input images are OOD** based on the uncertainty of the generation of the translated image.

6 BMC estimation

We now wish to apply the SURE uncertainty estimation method, as well as other approaches, to a specific task: the estimation of the Bone Mineral Content (BMC).

6.1 Task

Osteoporosis is a disease that causes bones to become fragile through a loss of tissue. It can be precisely diagnosed with the measurement of the bone mineral density (BMD). This measurement can be done using dual-energy X-rays or quantitative computed tomography (QCT), but they are expensive and not broadly available. It would therefore be interesting to be able to compute BMD using only X-ray images of the bones, which are a lot easier to obtain.

To do so, we aim at turning an X-ray image into a digitally reconstructed radiography (DRR) representing a QCT image. The BMD measurement could then be done using this DRR and traditional computations.



(a) Conditional image (b) Target image

Figure 13: Images involved in the translation

A pipeline using auto-encoders and a GAN already exists for this task [Gu23]. However, we are interesting in replacing it with a diffusion-based pipeline, given the very promising results of diffusion in a number of tasks, and so that we can apply our uncertainty metric.

In order to perform the image translation, we used a simple, already existing, pipeline: Palette [Sah+22]. Its principle is close to the one of a standard DDPM model, but it allows to concatenate a conditional image with the noised images used in the diffusion process, and input them in the denoising model. We modified the source code to implement a DDIM pipeline as well, following the same principle.

Once the DRR image is obtained, it is possible to compute the Bone Mineral Content (BMC), which is necessary to obtain the BMD, and on which we will focus in this work, using the following:

$$BMC = \sum_{(i,j) \in \llbracket 1,d \rrbracket^2} q_{i,j}$$

Where d is the dimension of the image and q represents the values of the pixels.

Before computing this quantity, the element sizes of the pixels of the condition images were used to correct the sizes of the generated images.

To assess the quality of the results, we used two correlation coefficients using the predicted BMCs and the ground truth ones: the Pearson correlation coefficient (PCC) and the Intraclass correlation coefficient (ICC). PCC is defined as:

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

Where cov designates the covariance and σ the standard deviation. It is equal to 1 if X and Y are perfectly correlated and is small otherwise.

ICC is defined as:

$$\iota_{X,Y} = 2 \frac{\mathbb{E}((X-m)(Y-m))}{\mathbb{E}((X-m)^2) + \mathbb{E}((Y-m)^2)}$$

Where $m = \frac{\mathbb{E}(X+Y)}{2}$. It is equal to 1 if X and Y are equal and small or negative if they are very different.

The results were aligned with the ground truth with an affine transformation before computing those metrics. A linear regression was performed, and the aligned predictions were computed using:

$$bmc_{pred-aligned} = \frac{(bmc_{pred} - intercept)}{slope}$$

We trained a Palette model on the same dataset as the one used in [Gu23] (separated in train and test sets of sizes 1989 and 662 respectively) for 760 epochs, with 10 diffusion steps, a cosine beta schedule, and images of size 128×256 . We also used data augmentation including translations, rotations, and scaling transformations. The conditional X-ray images were all individually normalized between 0 and 1 (because their absolute intensity does not bring valuable information), and the target DRR images were globally normalized using their global maximum intensity. Several optimizations of the diffusion process were made to improve the results:

- negative-mean: Since the mean of the target images is very close to 0 rather than 0.5, we tried starting the diffusion process with a noise of mean -0.1 rather than 0.
- altered clip: Usually the images are clipped between $[-1,1]$ during the diffusion process. However, since we wanted to get a background exactly equal to 0 in the generated images, we truncated between $[-1.05, 1.05]$ during the diffusion process, and only between $[-1,1]$ at its end.
- average: We thought that using the average of several generations with the same conditional image could yield better results than a single generation.

The results are the following:

generation method	PCC	ICC
average-altered clip-negative mean-DDIM	0.893	0.887
average-altered clip-negative mean-DDPM	0.892	0.886
altered clip-negative mean-DDPM	0.893	0.887
negative mean-DDPM	0.572	0.493
DDPM	0.02	0

Table 1: Results with Palette

These results confirm the relevance of our modifications. Here are scatter plots of the results:

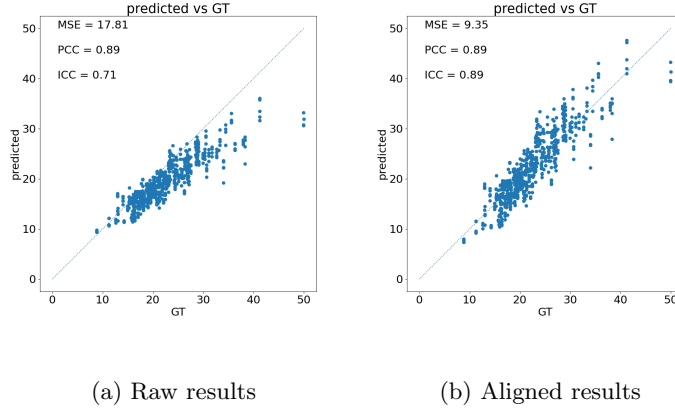


Figure 14: Scatter plot of the BMC prediction vs ground truth ; average-altered clip-negative mean-DDPM

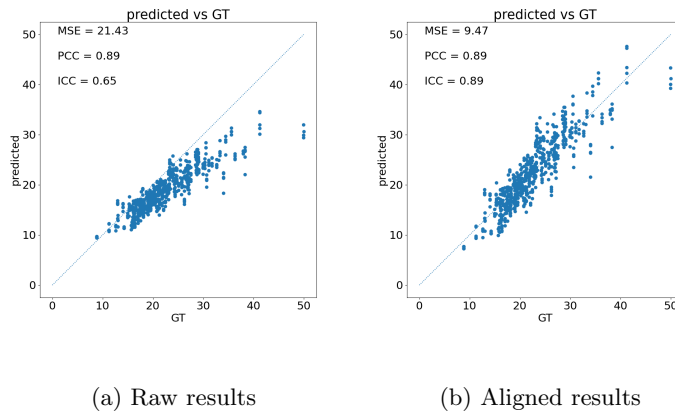


Figure 15: Scatter plot of the BMC prediction vs ground truth ; average-altered clip-negative mean-DDIM

We observe that results from DDIM are actually not as good as the ones from DDPM before alignment, but become very similar after.

The results are satisfying, especially since we are working with small images, but still far from the SOTA ([Gu23]), which are closer to PCC/ICC of 0.94. We tried our methods with bigger images of size 256×512 (580 epochs), and observed a small increase in accuracy.

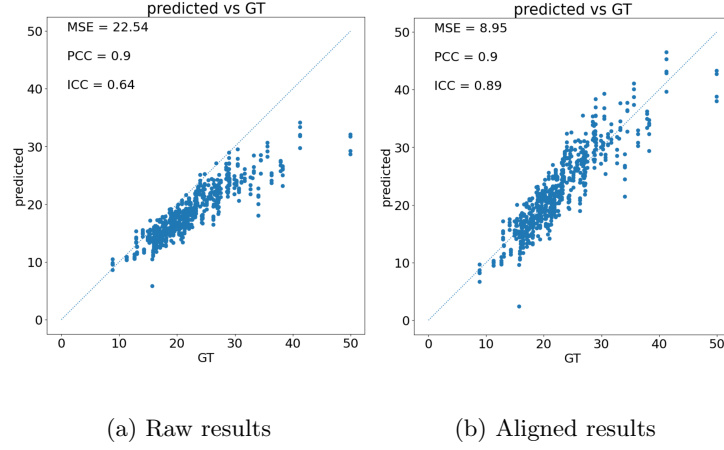


Figure 16: Scatter plot of the BMC prediction vs ground truth ; average-altered clip-negative mean-DDPM ; 256×512

However, the associated training and testing times skyrocketed. Therefore, to further improve the accuracy, it might be interesting to work with pipelines more advanced than Palette, and especially latent diffusion [Rom+22b]. This could allow for an increased precision with fast computation times.

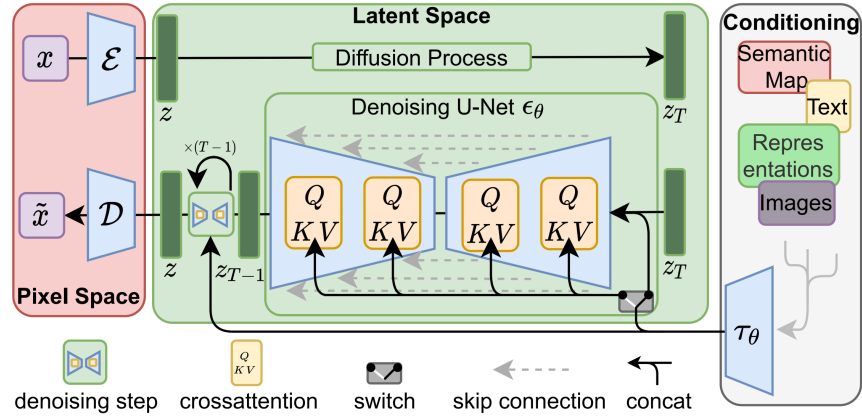


Figure 17: Latent diffusion diagram

It could even be possible to directly plug the encoder from [Gu23] in the conditioning block.

To try to improve the accuracy of the model, we also tried:

- Randomly modifying the luminosity of the conditional image during training.
- Giving more weight to the error on non-zero pixels for the training loss.
- Using more denoising steps.

None of these approaches yielded significantly improved results. It could also be interesting to try to use a MAE loss rather than a MSE one during training, since it is mentioned in [Sah+22] that such a loss promotes generation precision at the cost of diversity, which we are not interested in anyway.

6.2 Uncertainty metrics

We now wish to compute an uncertainty metric during the image generation, and assess whether or not it could allow to predict the error of the network when computing BMC.

6.2.1 Variance

Since the diffusion model learns $p(DRR|X-ray)$, computing the variance of the results over several generations for a single conditional image yields interesting insights about the certainty of the network with respect to the conditional image.

In general, this is not possible for diffusion models, because the probability distribution is closer to a gaussian mixture, with several different peaks, than to a single Gaussian. In such cases, it would be necessary to compute the entropy of the result rather than the variance, which is not necessarily possible. However, in our case, the optimal density should be a Dirac, since our problem is supposed to have a unique solution. Therefore it makes sense to compute the variance. This reasoning also applies to the other uncertainty metrics, and justifies using approaches over the whole diffusion process at once, rather than only ones that compile the results of independent computations over every denoising step.

6.2.2 SUREdiff

The SUREdiff algorithms described in this report can be applied directly. More precisely, we used three different SUREdiff-based approaches:

- Full-epsilon: We computed SUREdiff for every diffusion step, with $\epsilon = 0.01$ and $k = 5$. We multiplied the results, which accounts to computing the derivative of the final result with respect to the initial noise.
- Full-autograd: In the SUREdiff pipeline, instead of computing an approximation of the derivative over a region of the image, we computed the exact partial derivatives of the sum of the intensities of the pixels of the region using `autograd` from the `pytorch` library. Again, we supposed that faraway pixels were independent. We still used $k = 5$ and computed the product of the derivatives.
- Global: We also computed SUREdiff over the whole process at once, only computing the approximation of the derivative with the results of the full diffusion process. We controlled the random seeds of `pytorch` to make the reverse diffusion process deterministic, even with DDPM. We still used $k = 5$ and $\epsilon = 0.01$.

Because of the very small number of diffusion steps we used, the variance over the SURE computations is high and we sometimes got negative derivatives. We chose to clamp the SUREdiff images to positive values before using it for uncertainty computations.

6.2.3 MCDropOut

MCDropOut is a classical way of computing the uncertainty of a network trained using drop out. Drop out consists of randomly setting weights of the model to zero during the training process. MCDropOut consists of also randomly setting weight to zero during the generation process. [GG16] showed that, under some hypotheses of regularity, doing so allows to obtain a Bayesian model. Therefore, it is then possible to compute the variance of the results to get an uncertainty metric. In order to separate this method from the simple variance one, we controlled the random seeds of `pytorch` to always use the same noises in the reverse diffusion process.

6.2.4 BayesDiff (BD)

BayesDiff [Kou+24] is a method developed to estimate the uncertainty of a diffusion network that relies on last-layer Laplace approximation.

Laplace approximation is a method to approximate the distribution of the trained parameters of a network. It relies on the Taylor expansion of the log likelihood:

$$\log p(\theta|D) \approx \log p(\theta^*|D) + (\theta - \theta^*)^T J_\theta - \frac{1}{2}(\theta - \theta^*)^T H_\theta(\theta - \theta^*)$$

Where θ represents the parameters of the model, θ^* the optimal parameters, D the training set, J the Jacobian of the log likelihood and H its Hessian.

If the model is properly trained, $J_\theta \approx J_{\theta^*} \approx 0$. Therefore:

$$p(\theta|D) \approx p(\theta^*|D) + \exp\left(-\frac{1}{2}(\theta - \theta^*)^T H_\theta(\theta - \theta^*)\right)$$

and:

$$\theta \sim \mathcal{N}(\theta^*, H_\theta^{-1})$$

Since the Hessian can't usually be computed exactly because of the large number of parameters in deep learning networks, approximations are often used to compute H_θ^{-1} . For instance, last layer-approximation designates Laplace methods where the Hessian is only computed for the last layer of the network, which gives a decent approximation of the uncertainty of the output of the whole network [Dax+22]. Moreover, in the BayesDiff approach, the Hessian is considered positive semi definite, which allows us to leverage the Fisher information matrix and use instead of H [Dax+22]:

$$diag\left(\sum_{n=1}^N \mathbb{E}_{\hat{y} \sim (y|f_\theta(x_n))} [(\nabla_\theta \log p(\bar{y}|f_\theta(x_n))|\theta^*)(\nabla_\theta \log p(\hat{y}|f_\theta(x_n))|\theta^*)^T]\right)$$

where f is the model, $(x_n)_{n \in \llbracket 0, N \rrbracket}$ the training data, y the ground truth and $\hat{y}$ the model's prediction.

Last layer Laplace approximation therefore allows us to get an uncertainty for every model prediction. The BayesDiff paper then proposes an approach to propagate these uncertainties through the diffusion process, using the following algorithm:

Algorithm 1 Pixel-wise uncertainty estimation via Bayesian inference. (BayesDiff)**Input:** Starting point $\mathbf{x}_T$, Monte Carlo sample size S , Pre-trained noise prediction model ϵ_θ .**Output:** Image generation $\mathbf{x}_0$ and pixel-wise uncertainty $\text{Var}(\mathbf{x}_0)$.

- 1: Construct the pixel-wise variance prediction function γ_θ^2 via LLLA;
- 2: $\mathbb{E}(\mathbf{x}_T) \leftarrow \mathbf{x}_T$, $\text{Var}(\mathbf{x}_T) \leftarrow \mathbf{0}$, $\text{Cov}(\mathbf{x}_T, \epsilon_T) \leftarrow \mathbf{0}$;
- 3: **for** $t = T \rightarrow 1$ **do**
- 4: Sample $\epsilon_t \sim \mathcal{N}(\epsilon_\theta(\mathbf{x}_t, t), \text{diag}(\gamma_\theta^2(\mathbf{x}_t, t)))$;
- 5: Obtain $\mathbf{x}_{t-1}$ via Equation (7);
- 6: Estimate $\mathbb{E}(\mathbf{x}_{t-1})$ and $\text{Var}(\mathbf{x}_{t-1})$ via Equation (10) and Equation (8);
- 7: sample $\mathbf{x}_{t-1,i} \sim \mathcal{N}(\mathbb{E}(\mathbf{x}_{t-1}), \text{Var}(\mathbf{x}_{t-1}))$, $i = 1, \dots, S$;
- 8: Estimate $\text{Cov}(\mathbf{x}_{t-1}, \epsilon_{t-1})$ via Equation (11).
- 9: **end for**

Figure 18: BayesDiff algorithm

Where equation (7) is:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) + \sqrt{\beta_t} z$$

where $z \sim \mathcal{N}(0, \mathbb{I})$

Equation (10) is

$$\mathbb{E}(x_{t-1}) = \frac{1}{\sqrt{\alpha_t}} \mathbb{E}(x_t) - \frac{1 - \alpha_t}{\sqrt{(1 - \bar{\alpha}_t)\alpha_t}} \mathbb{E}(\epsilon_t)$$

Equation (8) is

$$\mathbb{V}(x_{t-1}) = \frac{1}{\alpha_t} \mathbb{V}(x_t) - 2 \frac{1 - \alpha_t}{\alpha_t \sqrt{1 - \bar{\alpha}_t}} \text{Cov}(x_t, \epsilon_t) + \frac{(1 - \alpha_t)^2}{\alpha_t (1 - \bar{\alpha}_t)} \mathbb{V}(\epsilon_t) + \beta_t$$

Equation (11) is

$$\text{Cov}(x_t, \epsilon_t) \approx \frac{1}{S} \sum_{i=1}^S (x_{t,i} \cdot \epsilon_\theta(x_{t,i}, t)) - \mathbb{E}x_t \cdot \frac{1}{S} \sum_{i=1}^S \epsilon_\theta(x_{t,i}, t)$$

where $\forall i \in \llbracket 1, S \rrbracket$, $x_{t,i} \sim \mathcal{N}(\mathbb{E}(x_t), \mathbb{V}(x_t))$.

Therefore, this method allows to get an estimate of the variance of the result of the diffusion process, which can serve as an uncertainty metric.

6.2.5 Image translation

Since our uncertainty methods output uncertainty images, it is also interesting to see if training a new Palette model using as conditions the X-Ray images and the predicted DRR image, and as target the prediction error, could yield good results.

6.2.6 Test time augmentation (TTA)

TTA is a traditionnal approach in computer vision [KSH12]. It consists in evaluating a model with several augmented inputs, e.g. translated or rotated versions of the original data. For every input, and if the model is

supposed to be invariant to the transformations used for the augmentation, the variance of the results can be computed and interpreted as a measure of the uncertainty of the network [AB18].

In our case, we applied translations, rotations and scaling to the conditional images and computed the results using the Palette network. For the pixel wise uncertainty, we applied the inverse transformations to the outputs of the model before computing the variance.

6.3 Entropy

We previously observed in this report the links between the uncertainty and the entropy of the images considered. One could expect that the error of the generation depends on the entropy of the condition image used. We therefore computed this entropy using `skimage`, and will compare it to the uncertainty maps produced with the other uncertainty approaches.

6.4 Results

For all the computations implying variances, 25 samples were used (Since SURE-based algorithms were used with $k = 5$, uncertainty methods require the equivalent of 25 or 26 generations - one generation takes about 1s). To get the image-level scores, we summed the values of the pixels of the uncertainty images corresponding to non-zero predicted values.

6.4.1 Pixel level

We are interested in the correlation between the pixel squared error, namely $(x_{predicted} - x_{GT})^2$ for every pixel of the generated image, and the uncertainty metric. We will use the mean PCC over all generations on our test set to quantify it.

uncertainty method	PCC
SURE global	0.08
SURE full epsilon	0.37
SURE full autograd	0.40
Variance	0.49
MCDropOut	0.49
BayesDiff	0.09
Palette	0.34
TTA	0.49
Intensity	0.15
Entropy	0.11

(a) DDPM

uncertainty method	PCC
SURE global	0.42
SURE full epsilon	0.40
SURE full autograd	0.40
Variance	0.47
MCDropOut	0.19
BayesDiff	0.09
TTA	0.47
Intensity	0.18
Entropy	0.11

(b) DDIM

Table 2: Uncertainty results at the pixel level

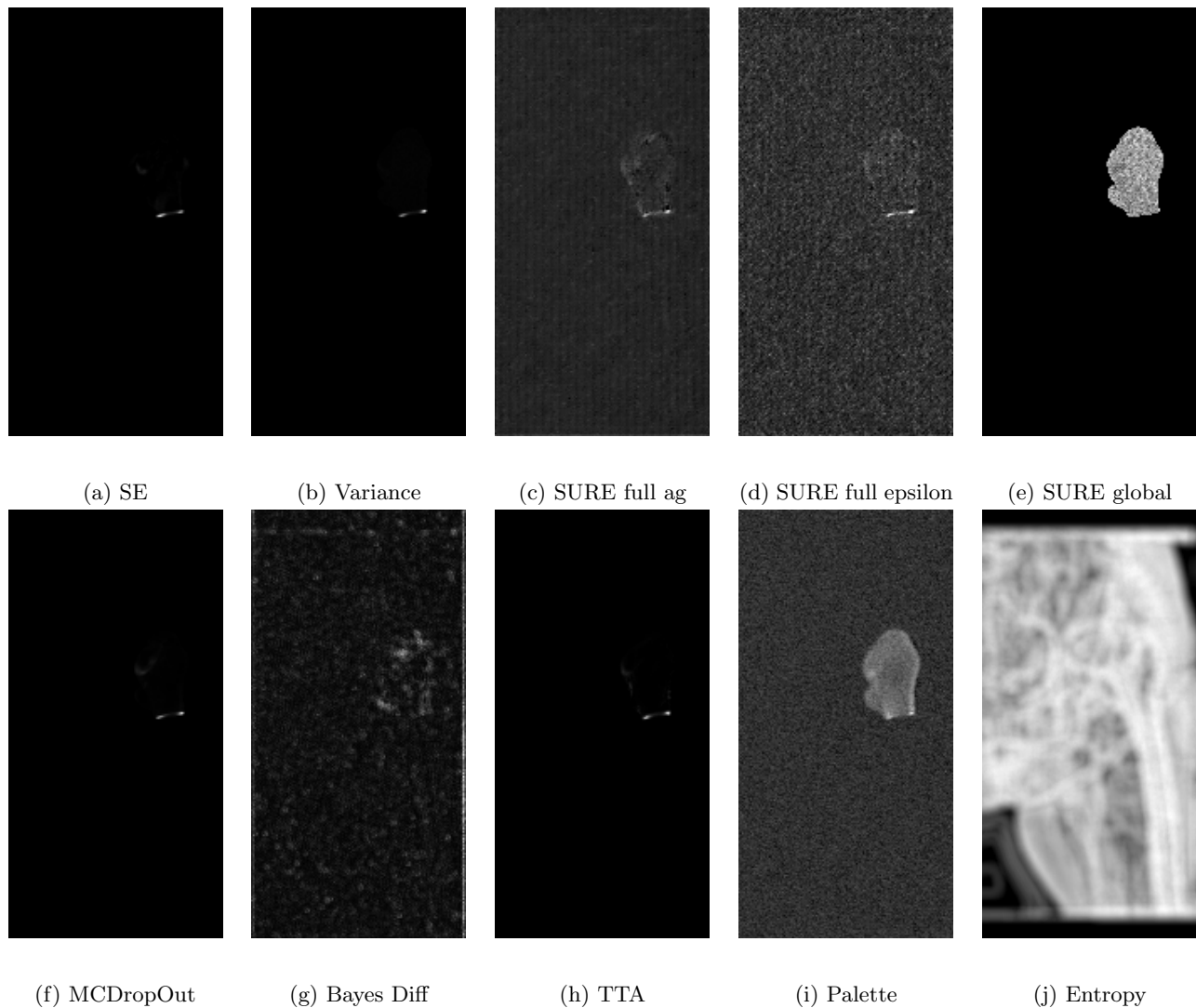


Figure 19: Uncertainty maps for DDPM

Here are visualizations of the results:

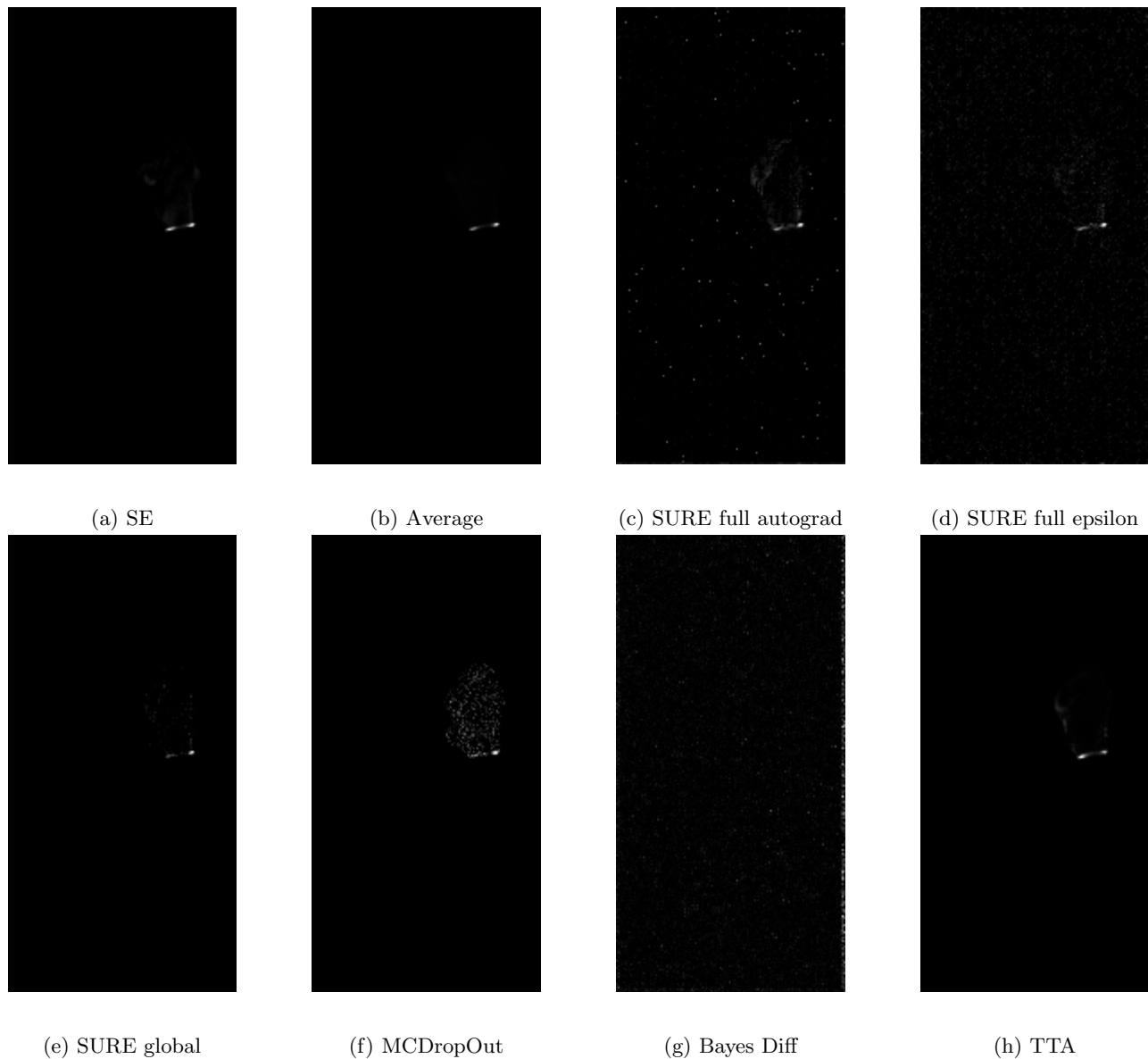


Figure 20: Uncertainty maps for DDIM

We observe that a good correlation is reached for the variance and the TTA approaches for DDPM and DDIM. With DDPM, MCDropOut also performs well, and with DDIM, SURE global does. These results suggest that it is possible to use these uncertainty metrics to detect badly generated areas in the images.

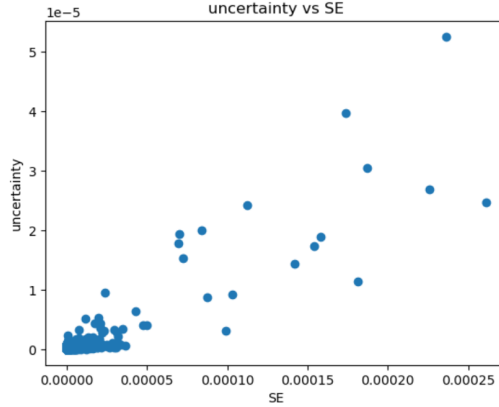


Figure 21: Scatter plot of the SE vs the uncertainty (variance) at the pixel level for *K4581_Abduction*

6.4.2 Image level

At the image level, we are interested in the correlation between the uncertainty and several quantities:

- the squared error (SE) of the generations, namely $\sum (x_{predicted} - x_{GT})^2$.
- the squared BMC prediction error (BMC SE), namely $(\sum x_{predicted} - \sum x_{GT})^2$.
- the relative squared BMC prediction error (BMC RSE), namely $(\sum x_{predicted} - \sum x_{GT})^2 / (\sum x_{GT})^2$.

Since the generations are usually corrected with an affine transformation to better match the ground truth, we will also investigate the correlation with:

- the corrected squared BMC prediction error (aligned BMC SE), namely $(\sum x_{corrected} - \sum x_{GT})^2$.
- the corrected relative squared BMC prediction error (aligned BMC RSE), namely $(\sum x_{corrected} - \sum x_{GT})^2 / (\sum x_{GT})^2$.

These quantities were derived from the results of a single generation only. By controlling the random seeds during the diffusion process, we made sure that the noises used for every approach were the same than the ones used for this generation.

6.4.3 Single generation

We were first interest in the correlation between the uncertainty and the error for the generation of a single image.

In order to quantify these correlations, we will use different metrics:

- AUC 95% (and 80%): the AUROC score of a classifier based on the order given by the uncertainties and that aims at separating the 5% (and 20%) of the predictions that have the highest error from the others.
- Spearman Correlation Coefficient (SCC): the Spearman rank-order correlation coefficient measures to what extent the orders defined by two variables are similar. It is interesting in our case because we don't

necessarily want the error and the uncertainty to be perfectly correlated, but we want them to have a monotonic relationship. It is defined by:

$$r_s = \frac{\text{Cov}(R(X), R(Y))}{\sigma_{R(X)} \sigma_{R(Y)}}$$

where σ is the standard deviation, and $R(X)$ is the ranks of X . The rank of an element of X is its index in the sorted list of the elements of X . The Spearman coefficient is equal to 1 if the orders are the same and very small otherwise.

The results are the following:

uncertainty method	SE			BMC SE			BMC RSE		
	AUC95	AUC80	SCC	AUC95	AUC80	SCC	AUC95	AUC80	SCC
SURE global	0.64	0.64	0.20	0.67	0.59	0.11	0.6	0.45	0
SURE full epsilon	0.67	0.64	0.22	0.68	0.62	0.17	0.61	0.50	0.04
SURE full autograd	0.59	0.58	0.13	0.58	0.57	0.14	0.55	0.51	0.06
Variance	0.77	0.72	0.35	0.82	0.66	0.16	0.73	0.50	0
MCDropOut	0.77	0.72	0.36	0.81	0.64	0.15	0.71	0.53	0.04
BayesDiff	0.63	0.66	0.25	0.70	0.63	0.17	0.55	0.48	0
Palette	0.61	0.64	0.19	0.65	0.63	0.14	0.56	0.46	0
TTA	0.70	0.67	0.25	0.74	0.63	0.11	0.62	0.48	0
Intensity	0.75	0.75	0.34	0.84	0.71	0.19	0.65	0.47	0
Entropy	0.60	0.62	0.16	0.62	0.62	0.13	0.55	0.48	0

Table 3: Uncertainty results at the image level - DDPM

uncertainty method	SE			BMC SE			BMC RSE		
	AUC95	AUC80	SCC	AUC95	AUC80	SCC	AUC95	AUC80	SCC
SURE global	0.83	0.77	0.33	0.86	0.70	0.23	0.78	0.60	0.07
SURE full epsilon	0.74	0.71	0.30	0.76	0.68	0.27	0.66	0.56	0.09
SURE full autograd	0.66	0.58	0.20	0.68	0.53	0.22	0.63	0.49	0.11
Variance	0.83	0.74	0.33	0.85	0.69	0.22	0.75	0.57	0.07
MCDropOut	0.80	0.68	0.26	0.79	0.67	0.19	0.69	0.53	0.03
BayesDiff	0.65	0.61	0.19	0.66	0.61	0.17	0.54	0.46	0
TTA	0.73	0.68	0.26	0.77	0.65	0.17	0.63	0.51	0.01
Intensity	0.80	0.76	0.35	0.84	0.74	0.29	0.64	0.51	0.01
Entropy	0.59	0.61	0.15	0.61	0.61	0.15	0.49	0.47	0

Table 4: Uncertainty results at the image level - DDIM

In the DDPM settings, the variance and MCDropOut approaches appear to be the most effective ones. In the DDIM settings, they yield even better results, but are outperformed by the SURE global approach. More generally, it seems that all the methods tend to perform better in the DDIM setting, which is probably due to the deterministic nature of the DDIM process.

uncertainty method	BMC SE			BMC RSE		
	AUC95	AUC80	SCC	AUC95	AUC80	SCC
SURE global	0.73	0.58	0.04	0.50	0.45	0
SURE full epsilon	0.75	0.60	0.06	0.54	0.47	0
SURE full autograd	0.68	0.57	0	0.55	0.48	0
Variance	0.86	0.70	0.23	0.64	0.55	0.08
MCDropOut	0.79	0.69	0.25	0.62	0.57	0.12
BayesDiff	0.68	0.54	0	0.43	0.44	-0.16
TTA	0.76	0.62	0.16	0.52	0.50	0.01
Intensity	0.83	0.66	0.17	0.4	0.44	0
Entropy	0.55	0.46	-0.11	0.46	0.47	-0.13

Table 5: Uncertainty results at the image level - DDPM - aligned

uncertainty method	BMC SE			BMC RSE		
	AUC95	AUC80	SCC	AUC95	AUC80	SCC
SURE global	0.85	0.67	0.25	0.60	0.55	0.10
SURE full epsilon	0.77	0.60	0.13	0.50	0.50	0
SURE full autograd	0.77	0.60	0.13	0.50	0.50	0
Variance	0.80	0.66	0.24	0.59	0.55	0.09
MCDropOut	0.82	0.62	0.14	0.57	0.51	0
BayesDiff	0.68	0.54	0	0.43	0.44	-0.16
TTA	0.75	0.59	0.14	0.48	0.48	0
Intensity	0.80	0.61	0.15	0.39	0.44	0
Entropy	0.52	0.46	-0.13	0.49	0.45	-0.15

Table 6: Uncertainty results at the image level - DDIM - aligned

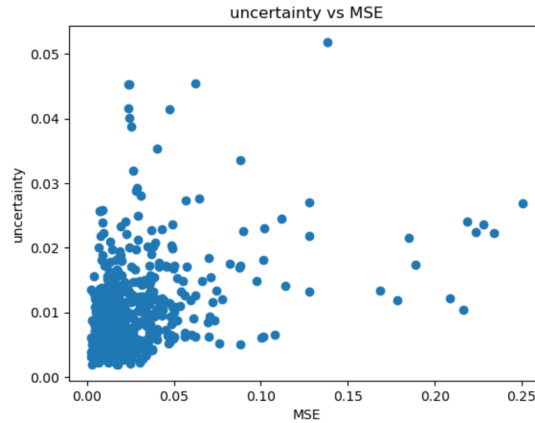


Figure 22: Scatter plot of the SE vs the uncertainty at the image level ; DDIM global

It is interesting to observe that simply taking the predicted pixel intensities as an uncertainty map yields results similar to the variance approach, both with DDPM and DDIM. This is probably specific to our dataset, in which the model is mostly trained on dark pixels and is therefore less certain when generating bright ones.

6.4.4 Mean of several generations

To try to achieve better correlations, we thought about using the mean error over several generations instead of only one generation. Indeed, theoretically, the SURE estimator is one of the expectancy of the error.

The results were the following, with errors averaged over 25 generations:

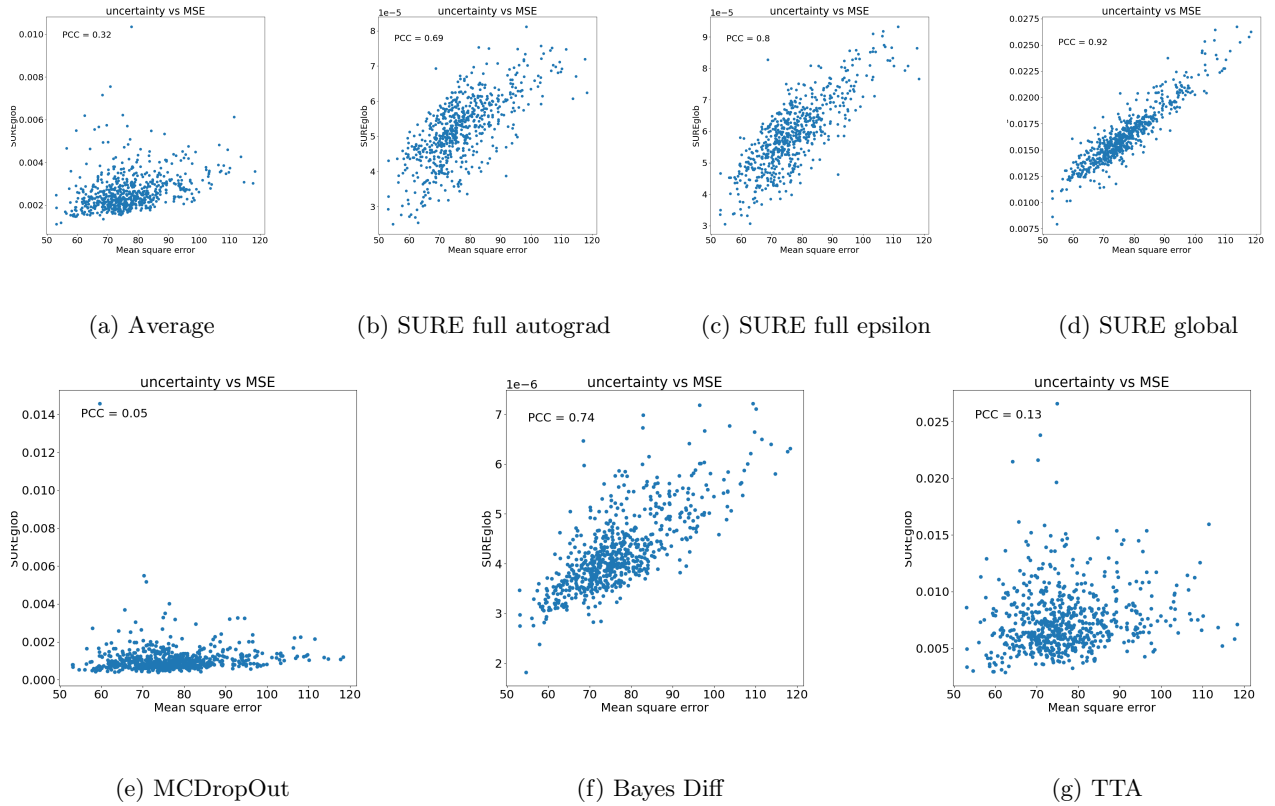


Figure 23: Uncertainty maps for DDIM

The results were improved a lot, the SURE methods yielding very promising correlations.

6.5 Failure detection

In practice, what we are interested in is the detection of failed generations. A generation is considered as a failure if the relative BMC prediction error is bigger than 20%. We will evaluate the effectiveness of the uncertainty approaches using the AUROC of a classifier aiming at detecting the failures. The results are the following:

uncertainty method	AUROC	AUROC aligned
SURE global	0.48	0.58
SURE full epsilon	0.52	0.58
SURE full autograd	0.51	0.56
Variance	0.55	0.64
MCDropOut	0.58	0.60
BayesDiff	0.47	0.54
TTA	0.48	0.53
Entropy	0.53	0.57

Table 7: Failure criteria - DDPM - number of failures: 79/661 non aligned, 86/661 aligned

uncertainty method	AUROC	AUROC aligned
SURE global	0.62	0.64
SURE full epsilon	0.56	0.58
SURE full autograd	0.48	0.51
Variance	0.59	0.64
MCDropOut	0.54	0.60
BayesDiff	0.46	0.56
TTA	0.51	0.58
Entropy	0.51	0.59

Table 8: Failure criteria - DDIM - number of failures: 170/661 non aligned, 86/661 aligned

Therefore, the uncertainties are not accurate enough to allow for useful failure detection. It is possible that using bigger images would allow for better results.

6.6 Accuracy improvement using the uncertainties

On top of trying to predict the model's errors with the uncertainties, it would be interesting to be able to improve the accuracy of the results using them. To do so, our idea is, for every conditional image, to filter several generations using the uncertainties to keep the best ones.

More precisely, we will compare:

- the average of 25 generations using a single conditional image and various noises during the reverse diffusion process.
- the average of only the 4 generations with smallest uncertainty.
- the generation with smallest uncertainty.

To compare the accuracy of the results, we will use the mean square error over the test dataset, as well as the PCC and ICC of the BMC prediction results. We chose the uncertainty metric that performed the best for the error prediction, namely SURE global in the DDIM settings, and got the following results:

The filtration therefore leads to a results' improvement. If it is non negligible for the raw predictions, it is, however, very small after aligning the results.

uncertainty method	MSE	PCC	ICC
Average	21.46	0.8928	0.6511
SURE global-filtered average 5	20.48	0.8924	0.6649
SURE global-filtered average 1	19.24	0.8931	0.6829

Table 9: Uncertainty results at the image level - DDIM

uncertainty method	MSE	PCC	ICC
Average	9.449	0.8928	0.8871
SURE global-filtered average 5	9.4964	0.8924	0.8867
SURE global-filtered average 1	9.426	0.8931	0.8874

Table 10: Uncertainty results at the image level after alignment with an affine transformation- DDIM

7 Conclusion

To conclude, this report describes **SUREdiff**, a new method to evaluate the uncertainty of the outputs of diffusion models based on the SURE estimator. SURE is a tool that allows one to access the expectancy of the MSE of a denoising estimator without requiring knowledge of the ground truth, the denoised data.

We highlighted the correlation between the intrinsic complexity of a generated image, its **local entropy**, and the uncertainty measurements. This phenomenon appears to be not specific to our method of uncertainty evaluation and might originate from the fact that a model is more uncertain when generating complex regions because they imply more individual choices, even if they are more probable than flat regions. This knowledge allowed us to improve our method by **normalizing the uncertainty** values for every entropy level of the generated image.

Using SUREdiff, it is possible to perform **OOD detection** by training a diffusion model on images following a reference distribution, and then using the SUREdiff values obtained when denoising test images to qualify them as OOD or ID. The first experiments we conducted using this method yielded very encouraging results. They showed that the method generalized sufficiently to learn semantic knowledge and detect ID samples following a distribution quite different from the training one. They also showed that if the distribution of ID images in training and testing phases was sufficiently similar, it was possible to get a perfect OOD classifier, even with very few training images. Tests still need to be done about **anomaly detection**, but the first results obtained are **promising**, and even seem to outperform a purely reconstruction-based OOD detection approach.

This estimator also benefits from **theoretical guarantees** that suggest that it could perform better with a diffusion model using smooth activation functions rather than ReLUs. This still has to be tested in practice.

Overall, we still have to evaluate our method for several tasks. Concerning OOD detection, it would be interesting to try our approach on classic OOD detection datasets and compare its results to other existing methods, especially Bayesian ones. With regards to uncertainty estimation at large, experiments with image translation should provide useful insights about the efficiency of SUREdiff. OOD detection of the input images according to the uncertainty of the output of the image translation model, and the study of the correlation between uncertainty values and the error of medical measurements based on generated images should especially be investigated. For all these tasks, comparing SUREdiff to the pixel-wise uncertainty estimation method for diffusion models described in [Kou+24], based on the last layer Laplace approximation, would also be very interesting. This is precisely what we will do during the rest of the internship, focusing first on the application to **image translation** with the settings of [Gu23].

8 References

- [Mac92] David John Cameron Mackay. *Bayesian methods for adaptive models*. California Institute of Technology, 1992.
- [Bis94] Christopher M Bishop. “Novelty detection and neural network validation”. In: *IEE Proceedings-Vision, Image and Signal processing* 141.4 (1994), pp. 217–222.
- [HD05] Aapo Hyvärinen and Peter Dayan. “Estimation of non-normalized statistical models by score matching.” In: *Journal of Machine Learning Research* 6.4 (2005).
- [GL06] Dani Gamerman and Hedibert F Lopes. *Markov chain Monte Carlo: stochastic simulation for Bayesian inference*. Chapman and Hall/CRC, 2006.
- [Vin11] Pascal Vincent. “A connection between score matching and denoising autoencoders”. In: *Neural computation* 23.7 (2011), pp. 1661–1674.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “Imagenet classification with deep convolutional neural networks”. In: *Advances in neural information processing systems* 25 (2012).
- [Van+14] Stefan Van der Walt et al. “scikit-image: image processing in Python”. In: *PeerJ* 2 (2014), e453.
- [RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. arXiv: 1505.04597.
- [GG16] Yarin Gal and Zoubin Ghahramani. *Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning*. 2016. arXiv: 1506.02142.
- [MW17] Amit Mandelbaum and Daphna Weinshall. *Distance-based Confidence Score for Neural Network Classifiers*. 2017. arXiv: 1709.09844.
- [AB18] Murat Seckin Ayhan and Philipp Berens. “Test-time data augmentation for estimation of heteroscedastic aleatoric uncertainty in deep neural networks”. In: *Medical Imaging with Deep Learning*. 2018.
- [HG18] Dan Hendrycks and Kevin Gimpel. *A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks*. 2018. arXiv: 1610.02136.
- [Kha+18] Mohammad Emtiyaz Khan et al. *Fast and Scalable Bayesian Deep Learning by Weight-Perturbation in Adam*. 2018. arXiv: 1806.04854.
- [Lee+18] Kimin Lee et al. *A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks*. 2018. arXiv: 1807.03888.
- [MG18] Andrey Malinin and Mark Gales. *Predictive Uncertainty Estimation via Prior Networks*. 2018. arXiv: 1802.10501.
- [RBB18] Hippolyt Ritter, Aleksandar Botev, and David Barber. “A Scalable Laplace Approximation for Neural Networks”. In: *International Conference on Learning Representations*. 2018. URL: <https://openreview.net/forum?id=Skdvd2xAZ>.
- [Blu+19] Hermann Blum et al. “Fishyscapes: A Benchmark for Safe Semantic Segmentation in Autonomous Driving”. In: *2019 IEEE/CVF International Conference on Computer Vision Workshop (IC-CVW)*. 2019, pp. 2403–2412. DOI: 10.1109/ICCVW.2019.00294.
- [Foo+19] Andrew YK Foong et al. “In-Between’Uncertainty in Bayesian Neural Networks”. In: *arXiv preprint arXiv:1906.11537* (2019).

- [Nal+19] Eric Nalisnick et al. “Do Deep Generative Models Know What They Don’t Know?” In: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=H1xwNhCcYm>.
- [Edu+20] Vineet Edupuganti et al. *Uncertainty Quantification in Deep MRI Reconstruction*. 2020. arXiv: 1901.11228.
- [HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239.
- [Lav+20] Max-Heinrich Laves et al. *Calibration of Model Uncertainty for Dropout Variational Inference*. 2020. arXiv: 2006.11584.
- [LLS20] Shiyu Liang, Yixuan Li, and R. Srikant. *Enhancing The Reliability of Out-of-distribution Image Detection in Neural Networks*. 2020. arXiv: 1706.02690.
- [Mal+20] Andrey Malinin et al. *Regression Prior Networks*. 2020. arXiv: 2006.11590.
- [SE20] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600.
- [Wan+20] Ziyu Wang et al. “Further analysis of outlier detection with deep generative models”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 8982–8992.
- [Wen+20] Florian Wenzel et al. *How Good is the Bayes Posterior in Deep Neural Networks Really?* 2020. arXiv: 2002.02405.
- [DN21] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233.
- [Son+21] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456.
- [Xie+21] Qin Xie et al. “Semisupervised training of deep generative models for high-dimensional anomaly detection”. In: *IEEE Transactions on Neural Networks and Learning Systems* 33.6 (2021), pp. 2444–2453.
- [Dax+22] Erik Daxberger et al. *Laplace Redux – Effortless Bayesian Deep Learning*. 2022. arXiv: 2106.14806 [cs.LG]. URL: <https://arxiv.org/abs/2106.14806>.
- [Kaw+22] Bahjat Kawar et al. *Denoising Diffusion Restoration Models*. 2022. arXiv: 2201.11793.
- [Lia+22] Yufei Liang et al. “Omni-frequency Channel-selection Representations for Unsupervised Anomaly Detection”. In: *arXiv preprint arXiv:2203.00259* (2022).
- [Pla+22] Patrick von Platen et al. *Diffusers: State-of-the-art diffusion models*. <https://github.com/huggingface/diffusers>. 2022.
- [Rom+22a] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752.
- [Rom+22b] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.
- [Sah+22] Chitwan Saharia et al. *Palette: Image-to-Image Diffusion Models*. 2022. arXiv: 2111.05826. URL: <https://arxiv.org/abs/2111.05826>.

- [Gu23] Yi Gu. “Bone mineral density estimation from a plain X-ray image by learning decomposition into projections of bone-segmented computed tomography”. In: *Medical Image Analysis* 90 (2023), p. 102970. ISSN: 1361-8415. DOI: <https://doi.org/10.1016/j.media.2023.102970>. URL: <https://www.sciencedirect.com/science/article/pii/S136184152300230X>.
- [Li+23] Bo Li et al. “Bbdm: Image-to-image translation with brownian bridge diffusion models”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern Recognition*. 2023, pp. 1952–1961.
- [BA24] Georges Le Bellier and Nicolas Audebert. *Detecting Out-Of-Distribution Earth Observation Images with Diffusion Models*. 2024. arXiv: 2404.12667.
- [Hu+24] Teng Hu et al. *AnomalyDiffusion: Few-Shot Anomaly Image Generation with Diffusion Model*. 2024. arXiv: 2312.05767.
- [Kou+24] Siqi Kou et al. *BayesDiff: Estimating Pixel-wise Uncertainty in Diffusion via Bayesian Inference*. 2024. arXiv: 2310.11142.
- [Yan+24] Jingkang Yang et al. *Generalized Out-of-Distribution Detection: A Survey*. 2024. arXiv: 2110.11334.

9 Appendix

Datasets visualization

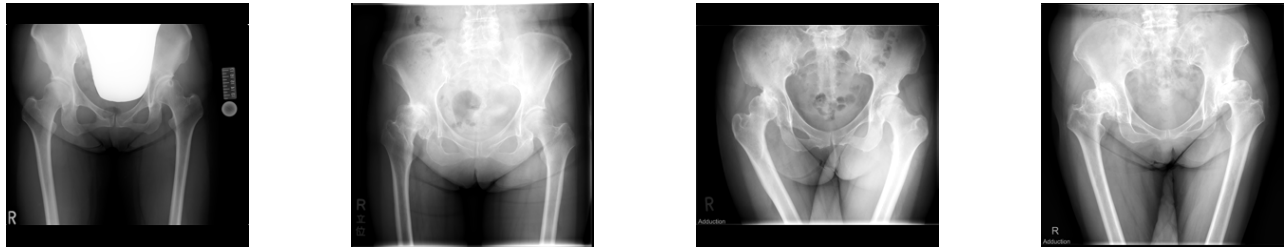


Figure 24: Images of hip x-ray from the 2652 images Osaka dataset

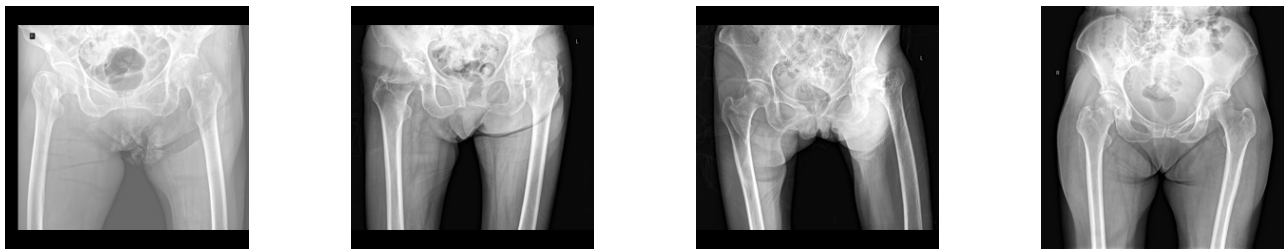


Figure 25: Images of hip x-ray from the 297 images dataset



Figure 26: Images of other x-rays from the 297 images dataset

SURE-related proofs

Expectancy

We keep the same notations as in 4.2. in the case where $x \in \mathbb{R}^d$.

Let's prove that

$$\mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(SURE_h(x)) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(\|\mu - h(x)\|^2) = \mathbb{E}_{x \sim \mathcal{N}(\mu, \sigma^2)}(MSE_h(x))$$

We have:

$$\begin{aligned} \mathbb{E}_x(\|\mu - h(x)\|^2) &= \mathbb{E}_x(\|g(x) + x - \mu\|^2) \\ &= \mathbb{E}_x(\|g(x)\|^2) + \mathbb{E}_x(\|x - \mu\|^2) + 2\mathbb{E}_x(g(x)^T(x - \mu)) \\ &= \mathbb{E}_x(\|g(x)\|^2) + d\sigma^2 + 2\mathbb{E}_x(g(x)^T(x - \mu)) \end{aligned}$$

Moreover, using integration by parts:

$$\begin{aligned} \mathbb{E}_x(g(x)^T(x - \mu)) &= \int_{\mathbb{R}^d} \frac{1}{\sqrt{2\pi\sigma^{2d}}} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma^2}\right) \sum_{i=1}^d g_i(x)(x_i - \mu_i) d^d x \\ &= \sigma^2 \sum_{i=1}^d \int_{\mathbb{R}^d} \frac{1}{\sqrt{2\pi\sigma^{2d}}} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma^2}\right) \frac{\partial g_i}{\partial x_i}(x) d^d x \\ &= \sigma^2 \sum_{i=1}^d \mathbb{E}_x\left(\frac{\partial g_i}{\partial x_i}(x)\right) \end{aligned}$$

Therefore:

$$\mathbb{E}_x(\|\mu - h(x)\|^2) = \mathbb{E}_x\left(d\sigma^2 + \|g(x)\|^2 + \sigma^2 \sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x)\right) = \mathbb{E}_x(SURE_h(x))$$

Variance

Let's now compute the variance of SURE. We have:

$$\mathbb{V}(SURE_h(x)) = \mathbb{E}((d\sigma^2 + \|g(x)\|^2 + 2\sigma^2 \sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x))^2) - \mathbb{E}(SURE_h(x))^2$$

And:

$$\begin{aligned} \mathbb{E}((d\sigma^2 + \|g(x)\|^2 + 2\sigma^2 Tr(Jac_g(x)))^2) &= \mathbb{E}(d^2\sigma^4 + \|g(x)\|^4 + 4\sigma^4 Tr(Jac_g(x))^2 + 2d\sigma^2\|g(x)\|^2 \\ &\quad + 4d\sigma^4 Tr(Jac_g(x)) + 4\sigma^2\|g(x)\|^2 Tr(Jac_g(x))) \end{aligned}$$

Moreover:

$$\mathbb{V}(MSE_h(x)) = \mathbb{E}(\|\mu - h(x)\|^2)^2 - \mathbb{E}(MSE_h(x))^2$$

With:

$$\begin{aligned} \mathbb{E}(\|\mu - h(x)\|^2)^2 &= \mathbb{E}(\|\mu - x - g(x)\|^2)^2 \\ &= \mathbb{E}(\|\mu - x\|^2 + \|g(x)\|^2 + 2g(x)^T(x - \mu))^2 \\ &= \mathbb{E}(\|x - \mu\|^4 + \|g(x)\|^4 + 2\|x - \mu\|^2\|g(x)\|^2 + 4g(x)^T(x - \mu)(\|x - \mu\|^2 + \|g(x)\|^2 + g(x)^T(x - \mu))) \end{aligned}$$

Let us compute all of these terms, using the fact that $\mathbb{E}_x(g(x)^T(x - \mu)) = \sigma^2 \sum_{i=1}^d \mathbb{E}_x(\frac{\partial g_i}{\partial x_i}(x))$ for any differentiable function g :

$$\begin{aligned} \mathbb{E}(\|x - \mu\|^4) &= (d + 2)\sigma^2 \mathbb{E}(\|x - \mu\|^2) \\ &= (d + 2)d\sigma^4 \\ \mathbb{E}(\|x - \mu\|^2\|g(x)\|^2) &= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial(\cdot - \mu_i)\|g(\cdot)\|^2}{\partial x_i}(x) \right) \\ &= \sigma^2 \mathbb{E} \left(d\|g(x)\|^2 + 2 \sum_{i=1}^d \frac{\partial g}{\partial x_i}(x)^T g(x)(x_i - \mu_i) \right) \\ &= \sigma^2 \mathbb{E} \left(d\|g(x)\|^2 + \sum_{i=1}^d \sum_{j=1}^d \frac{\partial g_j^2}{\partial x_i}(x)(x_i - \mu_i) \right) \\ &= \sigma^2 \mathbb{E} \left(d\|g(x)\|^2 + \left(\sum_{j=1}^d \frac{\partial g_j^2}{\partial x}(x) \right)^T (x - \mu) \right) \end{aligned}$$

Where:

$$\begin{aligned} \mathbb{E} \left(\left(\sum_{j=1}^d \frac{\partial g_j^2}{\partial x}(x) \right)^T (x - \mu) \right) &= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial \sum_{j=1}^d \frac{\partial g_j^2}{\partial x_i}(x)}{\partial x_i}(x) \right) \\ &= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2 g_j^2}{\partial x_i^2}(x) \right) \end{aligned}$$

$$\begin{aligned}
\mathbb{E}(g(x)^T(x - \mu)||x - \mu||^2) &= \sigma^2 \mathbb{E} \sum_{i=1}^d \left(\frac{\partial g_i ||\cdot - \mu||^2}{\partial x_i}(x) \right) \\
&= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \left(\frac{\partial g_i}{\partial x_i}(x) ||x - \mu||^2 + 2(x_i - \mu_i)g_i(x) \right) \right) \\
&= \sigma^2 \mathbb{E} \left(||x - \mu||^2 \sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) + 2g(x)^T(x - \mu) \right) \\
&= \mathbb{E} \left(d\sigma^4 \sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) + 2\sigma^4 \sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) \right) \\
&= (d + 2)\sigma^4 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) \right)
\end{aligned}$$

$$\begin{aligned}
\mathbb{E}(g(x)^T(x - \mu)||g(x)||^2) &= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i ||g(\cdot)||^2}{\partial x_i}(x) \right) \\
&= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) ||g(x)||^2 + 2g_i(x) \frac{\partial g}{\partial x_i}(x)^T g(x) \right) \\
&= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) ||g(x)||^2 \right) + 2\sigma^2 g(x)^T Jac_g(x)g(x)
\end{aligned}$$

$$\begin{aligned}
\mathbb{E}(g(x)^T(x - \mu)g(x)^T(x - \mu)) &= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i g(\cdot)^T(\cdot - \mu)}{\partial x_i}(x) \right) \\
&= \sigma^2 \mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i}{\partial x_i}(x) g(x)^T(x - \mu) + g_i(x) \left(\frac{\partial g}{\partial x_i}(x)^T(x - \mu) + g_i(x) \right) \right) \\
&= \sigma^2 \left(\mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i g}{\partial x_i}(x)^T(x - \mu) \right) + \mathbb{E}(||g(x)||^2) \right)
\end{aligned}$$

Where:

$$\begin{aligned}
\mathbb{E} \left(\sum_{i=1}^d \frac{\partial g_i g}{\partial x_i}(x)^T(x - \mu) \right) &= \sigma^2 \mathbb{E} \left(\sum_{j=1}^d \frac{\partial \sum_{i=1}^d \frac{\partial g_i g_j}{\partial x_i}}{\partial x_j}(x) \right) \\
&= \sigma^2 \mathbb{E} \left(\sum_{j=1}^d \sum_{i=1}^d \frac{\partial^2 g_i g_j}{\partial x_j \partial x_i}(x) \right)
\end{aligned}$$

Therefore:

$$\begin{aligned}
\mathbb{E}((\|\mu - h(x)\|^2)^2) &= \mathbb{E}(d^2\sigma^4 + 2d\sigma^4 + \|g(x)\|^4 + 2\sigma^2 d\|g(x)\|^2 + 2\sigma^4 \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2 g_j^2}{\partial x_i^2}(x) \\
&\quad + 4(d+2)\sigma^4 \text{Tr}(Jac_g(x)) + 8\sigma^2 g(x)^T Jac_g(x)g(x) + 4\sigma^2 \|g(x)\|^2 \text{Tr}(Jac_g(x)) \\
&\quad + 4\sigma^2 \|g(x)\|^2 + 4\sigma^4 \sum_{j=1}^d \sum_{i=1}^d \frac{\partial^2 g_i g_j}{\partial x_j \partial x_i}(x))
\end{aligned}$$

In the end, we have:

$$\begin{aligned}
\mathbb{V}(MSE_h(x)) - \mathbb{V}(SURE_h(x)) &= \mathbb{E}(-2d\sigma^4 + 4\sigma^2(2\sigma^2 \text{Tr}(Jac_g(x)) + \|g(x)\|^2 + d\sigma^2) + 8\sigma^2 g(x)^T Jac_g(x)g(x) \\
&\quad + 4\sigma^4 \sum_{j=1}^d \sum_{i=1}^d (\frac{\partial^2 g_i}{\partial x_j \partial x_i} g_j(x) + \frac{\partial^2 g_i}{\partial x_i \partial x_j} g_j(x) + \frac{\partial g_i}{\partial x_j} \frac{\partial g_j}{\partial x_i} + (\frac{\partial g_j}{\partial x_i})^2 + \frac{\partial^2 g_j}{\partial x_i^2} g_j(x))) \\
&= \mathbb{E}(-2d\sigma^4 + 4\sigma^2 SURE_h(x) + 8\sigma^2 g(x)^T Jac_g(x)g(x) + 2\sigma^4 \sum_{j=1}^d \sum_{i=1}^d (\frac{\partial g_i}{\partial x_j} + \frac{\partial g_j}{\partial x_i})^2 \\
&\quad + 4\sigma^4 \sum_{j=1}^d g_j(x) \sum_{i=1}^d (\frac{\partial^2 g_i}{\partial x_j \partial x_i}(x) + \frac{\partial^2 g_i}{\partial x_i \partial x_j}(x) + \frac{\partial^2 g_j}{\partial x_i^2}(x)))
\end{aligned}$$